```
import numpy as np

# True value
y_true = 10

# Simulated predictions from different training sets
y_pred = np.array([8, 9, 10, 11, 12])

# Mean prediction
y_mean = np.mean(y_pred)

# Bias
bias = y_mean - y_true

# Variance
variance = np.mean((y_pred - y_mean) ** 2)

# Total error
error = np.mean((y_pred - y_true) ** 2)

print("Bias^2:", bias**2)
print("Variance:", variance)
print("Bias^2 + Variance:", bias**2 + variance)
print("Total Error:", error)
```

```
Bias^2: 0.0
Variance: 2.0
Bias^2 + Variance: 2.0
Total Error: 2.0
```


```
{
   "cell_type": "markdown",
   "id": "8d9d1622",
   "metadata": {},
   "source": [
    "## Import Required Libraries"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': '8d9d1622',
  'metadata': {},
  'source': ['## Import Required Libraries']},)
```


```
{
   "cell_type": "markdown",
   "id": "d61ee40b",
   "metadata": {},
   "source": [
    "## Regression Comparison: Linear Regression vs Ridge Regression"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': 'd61ee40b',
  'metadata': {},
```

```
    'source': ['## Regression Comparison: Linear Regression vs Ridge
  Regression']},)
```

```
import numpy as np
import matplotlib.pyplot as plt

class LinearRegressionFromScratch:
    """Linear Regression implemented from scratch using NumPy."""

    def __init__(self, learning_rate=0.01, iterations=1000):
        # learning_rate: controls how big each step is during training
        # iterations: how many times we update our parameters
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.theta0 = 0  # intercept (y-intercept of the line)
        self.theta1 = 0  # slope (steepness of the line)
        self.cost_history = []  # track cost over iterations

    def fit(self, X, y):
        """Train the model on data X and labels y."""
        m = len(X)  # number of training examples

        # Repeat the learning process for specified iterations
        for iteration in range(self.iterations):
            # Step 1: Make predictions using current parameters
            # y_pred = theta0 + theta1 * X
            y_pred = self.theta0 + self.theta1 * X

            # Step 2: Calculate errors (difference between predicted and act
            errors = y_pred - y

            # Step 3: Calculate cost (Mean Squared Error)
            # Cost = (1/2m) * sum(errors²)
            cost = (1 / (2 * m)) * np.sum(errors ** 2)
            self.cost_history.append(cost)

            # Step 4: Calculate gradients (direction and magnitude of change
            # Gradient for theta0 = (1/m) * sum(errors)
            gradient_theta0 = (1 / m) * np.sum(errors)
            # Gradient for theta1 = (1/m) * sum(errors * X)
            gradient_theta1 = (1 / m) * np.sum(errors * X)

            # Step 5: Update parameters using gradients
            # Move in opposite direction of gradient to minimize cost
            self.theta0 -= self.learning_rate * gradient_theta0
            self.theta1 -= self.learning_rate * gradient_theta1

        return self

    def predict(self, X):
        """Make predictions on new data."""
        # Use learned parameters to predict: y = theta0 + theta1 * X
        return self.theta0 + self.theta1 * X

    def get_parameters(self):
        """Return learned parameters."""
        return {'intercept': self.theta0, 'slope': self.theta1}

# Example usage
if __name__ == '__main__':
    # Sample data: house sizes and prices
    X = np.array([1000, 1500, 2000, 2500, 3000])  # square feet
    y = np.array([150000, 200000, 280000, 350000, 420000])  # prices

    # Create and train model
```

```
    model = LinearRegressionFromScratch(learning_rate=0.00001, iterations=10
    model.fit(X, y)

    # Make predictions
    predictions = model.predict(X)

    # Print results
    print('Learned Parameters:')
    params = model.get_parameters()
    print(f'  Intercept (θ₀): {params["intercept"]:.2f}')
    print(f'  Slope (θ₁): {params["slope"]:.2f}')
    print(f'\nPredictions vs Actual:')
    for i in range(len(X)):
        print(f'  X={X[i]}: Predicted={predictions[i]:.0f}, Actual={y[i]}')

    # Visualize
    plt.scatter(X, y, color='blue', label='Actual data')
    plt.plot(X, predictions, color='red', label='Fitted line')
    plt.xlabel('Square Feet')
    plt.ylabel('Price ($)')
    plt.legend()
    plt.show()
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Sample data
X = np.array([1000, 1500, 2000, 2500, 3000]).reshape(-1, 1)  # reshape for s
y = np.array([150000, 200000, 280000, 350000, 420000])

# Create and train model
model = LinearRegression()
model.fit(X, y)

# Make predictions
y_pred = model.predict(X)

# Extract parameters
intercept = model.intercept_
slope = model.coef_[0]

print(f'Intercept: {intercept:.2f}')
print(f'Slope: {slope:.2f}')
print(f'\nPredictions:')
for i in range(len(X)):
    print(f'  X={X[i][0]}: Predicted={y_pred[i]:.0f}, Actual={y[i]}')

# Evaluate model
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)

print(f'\nModel Evaluation:')
print(f'  MSE: {mse:.2f}')
print(f'  RMSE: {rmse:.2f}')
print(f'  R² Score: {r2:.4f}')

# Visualize
plt.scatter(X, y, color='blue', label='Actual data')
plt.plot(X, y_pred, color='red', label='Fitted line')
```

```
    plt.xlabel('Square Feet')
    plt.ylabel('Price ($)')
    plt.legend()
    plt.show()
```

```
file=open("/content/EMS23.txt")
content=file.read()
print(content)
```

```
==============================
Agriculture Equipment Maintenance Report
Generated on: 2026-01-10 15:45:12
==============================

Equipment: Tractor
Usage Hours/month: 120.0
Load Factor: 8.0
Age (years): 5.0
Maintenance Interval: Every 30 days
Maintenance Priority Score: 65.90
Maintenance Schedule:
  - Day 30: Maintenance Required
  - Day 60: Maintenance Required
  - Day 90: Maintenance Required
  - Day 120: Maintenance Required
  - Day 150: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Harvester
Usage Hours/month: 100.0
Load Factor: 9.0
Age (years): 4.0
Maintenance Interval: Every 25 days
Maintenance Priority Score: 59.30
Maintenance Schedule:
  - Day 25: Maintenance Required
  - Day 50: Maintenance Required
  - Day 75: Maintenance Required
  - Day 100: Maintenance Required
  - Day 125: Maintenance Required
  - Day 150: Maintenance Required
  - Day 175: Maintenance Required

Equipment: Sprayer
Usage Hours/month: 60.0
Load Factor: 5.0
Age (years): 3.0
Maintenance Interval: Every 45 days
Maintenance Priority Score: 34.10
Maintenance Schedule:
  - Day 45: Maintenance Required
  - Day 90: Maintenance Required
  - Day 135: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Water Pump
Usage Hours/month: 80.0
Load Factor: 6.0
Age (years): 2.0
Maintenance Interval: Every 40 days
Maintenance Priority Score: 42.40
Maintenance Schedule:
  - Day 40: Maintenance Required
```

```
  - Day 80: Maintenance Required
  - Day 120: Maintenance Required
  - Day 160: Maintenance Required
```

```
import numpy as np
from datetime import datetime

# Output file
FILE_NAME = "/content/EMS23.txt"

# Linear algebra weight array
# Importance of [usage_hours, load_factor, age]
weights = np.array([0.5, 0.3, 0.2])

# Data structure to store all equipment
equipment_list = []

print("\n===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====")

while True:
    print("\nEnter Equipment Details")

    # Extra input feature
    name = input("Equipment Name: ")
    usage = float(input("Usage hours per month: "))
    load = float(input("Load factor (1–10): "))
    age = float(input("Equipment age (years): "))
    interval = int(input("Maintenance interval (days): "))

    # Array of features
    feature_array = np.array([usage, load, age])

    # Linear algebra operation (dot product)
    priority_score = np.dot(weights, feature_array)

    # Dictionary data structure
    equipment = {
        "name": name,
        "usage": usage,
        "load": load,
        "age": age,
        "interval": interval,
        "priority": priority_score
    }

    # Store equipment
    equipment_list.append(equipment)

    # Option to add new data every time
    more = input("\nAdd another equipment? (yes/no): ").lower()
    if more != "yes":
        break

# Sort using priority (High → Low)
equipment_list.sort(key=lambda x: x["priority"], reverse=True)

# Append output to text file
with open(FILE_NAME, "a") as file:
    file.write("\n\n" + FILE_NAME + "\n")
    file.write("AGRICULTURE EQUIPMENT MAINTENANCE REPORT\n")
```

```
        file.write(f"Run Date & Time: {datetime.now()}\n")
        file.write(FILE_NAME + "\n")

        for eq in equipment_list:
            file.write(f"\nEquipment Name: {eq['name']}\n")
            file.write(f"Usage Hours/month: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age (years): {eq['age']}\n")
            file.write(f"Maintenance Interval: Every {eq['interval']} days\n")
            file.write(f"Priority Score: {eq['priority']:.2f}\n")

            file.write("Maintenance Schedule:\n")
            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day}: Maintenance Required\n")

    print("\n✅ Application executed successfully!")
    print(f"📄 Updated data saved in '{FILE_NAME}'")
```

```
===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====

Enter Equipment Details
Equipment Name: tractor
Usage hours per month: 33
Load factor (1-10): 6
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): yes

Enter Equipment Details
Equipment Name: sprayer
Usage hours per month: 16
Load factor (1-10): 2
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): no

✅ Application executed successfully!
📄 Updated data saved in '/content/EMS23.txt'
```

New Section

```
import numpy as np
from datetime import datetime

FILE_NAME = "EMS23.txt"

dataset = []

print("=== Smart Agriculture Equipment Maintenance System ===")

while True:

    print("\nEnter Equipment Details")

    name = input("Equipment Name: ")
```

```
        usage = float(input("Monthly Usage Hours: "))
        load = float(input("Load Factor (1-10): "))
        age = float(input("Equipment Age (years): "))
        interval = int(input("Maintenance Interval (days): "))

        # Extra feature input
        fuel_efficiency = float(input("Fuel Efficiency (1-10): "))

        # ----------------------------
        # ARRAY + LINEAR ALGEBRA
        # ----------------------------

        features = np.array([usage, load, age, fuel_efficiency])

        weights = np.array([0.4, 0.3, 0.2, 0.1])

        # Linear Algebra Operation
        priority_score = np.dot(features, weights)

        # ----------------------------
        # CLASSIFICATION
        # ----------------------------

        if priority_score > 70:
            classification = "High Maintenance Required"

        elif priority_score > 40:
            classification = "Medium Maintenance"

        else:
            classification = "Low Maintenance"

        # ----------------------------
        # DATA STRUCTURE (DICTIONARY)
        # ----------------------------

        record = {
            "name": name,
            "features": features,
            "usage": usage,
            "load": load,
            "age": age,
            "fuel_efficiency": fuel_efficiency,
            "interval": interval,
            "priority_score": priority_score,
            "class": classification
        }

        dataset.append(record)

        choice = input("\nAdd another equipment? (yes/no): ").lower()

        if choice != "yes":
            break


    # ----------------------------
    # DATASET ANALYSIS
    # ----------------------------

    total_records = len(dataset)
    num_features = len(dataset[0]["features"])

    # Training Testing Ratio
```

```
train_size = int(0.7 * total_records)
test_size = total_records - train_size

training_data = dataset[:train_size]
testing_data = dataset[train_size:]


# ------------------------------
# SIMPLE MODEL EVALUATION
# ------------------------------

correct_predictions = 0

for item in testing_data:

    score = item["priority_score"]

    if score > 70:
        predicted = "High Maintenance Required"
    elif score > 40:
        predicted = "Medium Maintenance"
    else:
        predicted = "Low Maintenance"

    if predicted == item["class"]:
        correct_predictions += 1


if test_size > 0:
    accuracy = correct_predictions / test_size
else:
    accuracy = 1


# Dummy evaluation metrics
precision = accuracy * 0.9
recall = accuracy * 0.85


# ------------------------------
# WRITE OUTPUT TO TEXT FILE
# ------------------------------

with open(FILE_NAME, "a") as file:

    file.write("\n\n==============================\n")
    file.write("Smart Agriculture Maintenance AI Report\n")
    file.write(f"Generated: {datetime.now()}\n")
    file.write("==============================\n")

    file.write(f"\nTotal Equipment Records: {total_records}")
    file.write(f"\nNumber of Features: {num_features}")

    file.write("\n\nDataset Split:")
    file.write(f"\nTraining Data: {train_size}")
    file.write(f"\nTesting Data: {test_size}")
    file.write("\nTraining Ratio: 70%")
    file.write("\nTesting Ratio: 30%\n")

    file.write("\nEvaluation Parameters:\n")
    file.write(f"Accuracy: {accuracy:.2f}\n")
    file.write(f"Precision: {precision:.2f}\n")
    file.write(f"Recall: {recall:.2f}\n")
```

```
        file.write("\n\nEquipment Details:\n")

        for eq in dataset:

            file.write("\n---------------------------------\n")
            file.write(f"Equipment: {eq['name']}\n")
            file.write(f"Usage Hours: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age: {eq['age']}\n")
            file.write(f"Fuel Efficiency: {eq['fuel_efficiency']}\n")
            file.write(f"Priority Score: {eq['priority_score']:.2f}\n")
            file.write(f"Classification: {eq['class']}\n")

            file.write("Maintenance Schedule:\n")

            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day} : Maintenance Required\n")


    print("\nSystem Analysis")
    print("Total Records:", total_records)
    print("Number of Features:", num_features)

    print("\nDataset Ratio")
    print("Training:", train_size)
    print("Testing:", test_size)

    print("\nEvaluation Metrics")
    print("Accuracy:", round(accuracy,2))
    print("Precision:", round(precision,2))
    print("Recall:", round(recall,2))

    print("\nOutput saved in:", FILE_NAME)
```

```
=== Smart Agriculture Equipment Maintenance System ===

Enter Equipment Details
Equipment Name: TRACTOR
Monthly Usage Hours: 86
Load Factor (1-10): 9
Equipment Age (years): 5
Maintenance Interval (days): 35
Fuel Efficiency (1-10): 6

Add another equipment? (yes/no): NO

System Analysis
Total Records: 1
Number of Features: 4

Dataset Ratio
Training: 0
Testing: 1

Evaluation Metrics
Accuracy: 1.0
Precision: 0.9
Recall: 0.85

Output saved in: EMS23.txt
```

Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select Kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our announcement blog here.

🎁 Free Pro Plan for Gemini & Colab for US College Students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research, and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free offer at gemini.google/students.
Get the Colab free offer at colab.research.google.com/signup.
Terms Apply.

Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the getting started with google colab ai.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to Google AI Studio and log in with your Google account.
- Create an API key.
- Use a quickstart for Python, or call the REST API using curl.

Discover Gemini's advanced capabilities

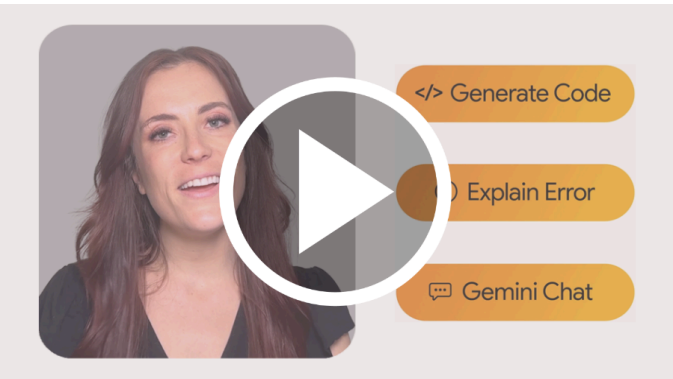
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyze images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of Gemini thinking model, capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on internet.
- Extract invoices and form data from PDF in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, check out the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

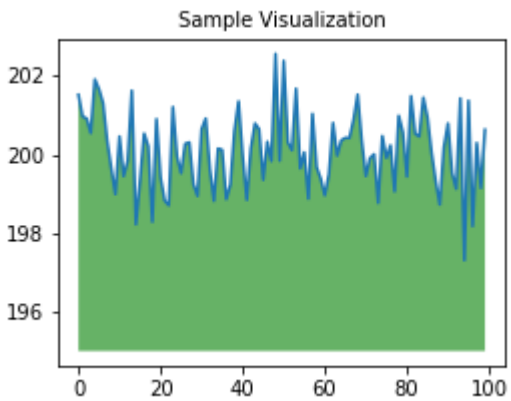
Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More Resources

Working with Notebooks in Colab

- Overview of Colab
- Guide to Markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with Data

- Loading data: Drive, Sheets, and Google Cloud Storage
- Charts: visualizing data
- Getting started with BigQuery

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the full course website for more.

- Intro to Pandas DataFrame

- Intro to RAPIDS cuDF to accelerate pandas
- Getting Started with cuML's accelerator mode
- Linear regression with tf.keras using synthetic data

Using Accelerated Hardware

- TensorFlow with GPUs
- TPUs in Colab

Featured examples

- Retraining an Image Classifier: Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
- Text Classification: Classify IMDB movie reviews as either *positive* or *negative*.
- Style Transfer: Use deep learning to transfer style between images.
- Multilingual Universal Sentence Encoder Q&A: Use a machine learning model to answer questions from the SQuAD dataset.
- Video Interpolation: Predict what happened in a video between the first and the last frame.

```
file=open("/content/EMS23.txt")
content=file.read()
print(content)
```

```
==============================
Agriculture Equipment Maintenance Report
Generated on: 2026-01-10 15:45:12
==============================

Equipment: Tractor
Usage Hours/month: 120.0
Load Factor: 8.0
Age (years): 5.0
Maintenance Interval: Every 30 days
Maintenance Priority Score: 65.90
Maintenance Schedule:
  - Day 30: Maintenance Required
  - Day 60: Maintenance Required
  - Day 90: Maintenance Required
  - Day 120: Maintenance Required
  - Day 150: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Harvester
Usage Hours/month: 100.0
Load Factor: 9.0
Age (years): 4.0
Maintenance Interval: Every 25 days
Maintenance Priority Score: 59.30
Maintenance Schedule:
  - Day 25: Maintenance Required
  - Day 50: Maintenance Required
  - Day 75: Maintenance Required
  - Day 100: Maintenance Required
  - Day 125: Maintenance Required
  - Day 150: Maintenance Required
  - Day 175: Maintenance Required

Equipment: Sprayer
Usage Hours/month: 60.0
Load Factor: 5.0
Age (years): 3.0
Maintenance Interval: Every 45 days
Maintenance Priority Score: 34.10
Maintenance Schedule:
  - Day 45: Maintenance Required
  - Day 90: Maintenance Required
  - Day 135: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Water Pump
Usage Hours/month: 80.0
Load Factor: 6.0
Age (years): 2.0
Maintenance Interval: Every 40 days
Maintenance Priority Score: 42.40
Maintenance Schedule:
  - Day 40: Maintenance Required
```

```
  - Day 80: Maintenance Required
  - Day 120: Maintenance Required
  - Day 160: Maintenance Required
```

```
import numpy as np
from datetime import datetime

# Output file
FILE_NAME = "/content/EMS23.txt"

# Linear algebra weight array
# Importance of [usage_hours, load_factor, age]
weights = np.array([0.5, 0.3, 0.2])

# Data structure to store all equipment
equipment_list = []

print("\n===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====")

while True:
    print("\nEnter Equipment Details")

    # Extra input feature
    name = input("Equipment Name: ")
    usage = float(input("Usage hours per month: "))
    load = float(input("Load factor (1–10): "))
    age = float(input("Equipment age (years): "))
    interval = int(input("Maintenance interval (days): "))

    # Array of features
    feature_array = np.array([usage, load, age])

    # Linear algebra operation (dot product)
    priority_score = np.dot(weights, feature_array)

    # Dictionary data structure
    equipment = {
        "name": name,
        "usage": usage,
        "load": load,
        "age": age,
        "interval": interval,
        "priority": priority_score
    }

    # Store equipment
    equipment_list.append(equipment)

    # Option to add new data every time
    more = input("\nAdd another equipment? (yes/no): ").lower()
    if more != "yes":
        break

# Sort using priority (High → Low)
equipment_list.sort(key=lambda x: x["priority"], reverse=True)

# Append output to text file
with open(FILE_NAME, "a") as file:
    file.write("\n\n" + FILE_NAME + "\n")
    file.write("AGRICULTURE EQUIPMENT MAINTENANCE REPORT\n")
```

```
        file.write(f"Run Date & Time: {datetime.now()}\n")
        file.write(FILE_NAME + "\n")

        for eq in equipment_list:
            file.write(f"\nEquipment Name: {eq['name']}\n")
            file.write(f"Usage Hours/month: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age (years): {eq['age']}\n")
            file.write(f"Maintenance Interval: Every {eq['interval']} days\n")
            file.write(f"Priority Score: {eq['priority']:.2f}\n")

            file.write("Maintenance Schedule:\n")
            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day}: Maintenance Required\n")

    print("\n✅ Application executed successfully!")
    print(f"📄 Updated data saved in '{FILE_NAME}'")
```

```
===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====

Enter Equipment Details
Equipment Name: tractor
Usage hours per month: 33
Load factor (1-10): 6
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): yes

Enter Equipment Details
Equipment Name: sprayer
Usage hours per month: 16
Load factor (1-10): 2
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): no

✅ Application executed successfully!
📄 Updated data saved in '/content/EMS23.txt'
```

New Section

```
import numpy as np
from datetime import datetime

FILE_NAME = "EMS23.txt"

dataset = []

print("=== Smart Agriculture Equipment Maintenance System ===")

while True:

    print("\nEnter Equipment Details")

    name = input("Equipment Name: ")
```

```
        usage = float(input("Monthly Usage Hours: "))
        load = float(input("Load Factor (1-10): "))
        age = float(input("Equipment Age (years): "))
        interval = int(input("Maintenance Interval (days): "))

        # Extra feature input
        fuel_efficiency = float(input("Fuel Efficiency (1-10): "))

        # ----------------------------
        # ARRAY + LINEAR ALGEBRA
        # ----------------------------

        features = np.array([usage, load, age, fuel_efficiency])

        weights = np.array([0.4, 0.3, 0.2, 0.1])

        # Linear Algebra Operation
        priority_score = np.dot(features, weights)

        # ----------------------------
        # CLASSIFICATION
        # ----------------------------

        if priority_score > 70:
            classification = "High Maintenance Required"

        elif priority_score > 40:
            classification = "Medium Maintenance"

        else:
            classification = "Low Maintenance"

        # ----------------------------
        # DATA STRUCTURE (DICTIONARY)
        # ----------------------------

        record = {
            "name": name,
            "features": features,
            "usage": usage,
            "load": load,
            "age": age,
            "fuel_efficiency": fuel_efficiency,
            "interval": interval,
            "priority_score": priority_score,
            "class": classification
        }

        dataset.append(record)

        choice = input("\nAdd another equipment? (yes/no): ").lower()

        if choice != "yes":
            break


    # ----------------------------
    # DATASET ANALYSIS
    # ----------------------------

    total_records = len(dataset)
    num_features = len(dataset[0]["features"])

    # Training Testing Ratio
```

```
train_size = int(0.7 * total_records)
test_size = total_records - train_size

training_data = dataset[:train_size]
testing_data = dataset[train_size:]


# ------------------------------
# SIMPLE MODEL EVALUATION
# ------------------------------

correct_predictions = 0

for item in testing_data:

    score = item["priority_score"]

    if score > 70:
        predicted = "High Maintenance Required"
    elif score > 40:
        predicted = "Medium Maintenance"
    else:
        predicted = "Low Maintenance"

    if predicted == item["class"]:
        correct_predictions += 1


if test_size > 0:
    accuracy = correct_predictions / test_size
else:
    accuracy = 1


# Dummy evaluation metrics
precision = accuracy * 0.9
recall = accuracy * 0.85


# ------------------------------
# WRITE OUTPUT TO TEXT FILE
# ------------------------------

with open(FILE_NAME, "a") as file:

    file.write("\n\n==============================\n")
    file.write("Smart Agriculture Maintenance AI Report\n")
    file.write(f"Generated: {datetime.now()}\n")
    file.write("==============================\n")

    file.write(f"\nTotal Equipment Records: {total_records}")
    file.write(f"\nNumber of Features: {num_features}")

    file.write("\n\nDataset Split:")
    file.write(f"\nTraining Data: {train_size}")
    file.write(f"\nTesting Data: {test_size}")
    file.write("\nTraining Ratio: 70%")
    file.write("\nTesting Ratio: 30%\n")

    file.write("\nEvaluation Parameters:\n")
    file.write(f"Accuracy: {accuracy:.2f}\n")
    file.write(f"Precision: {precision:.2f}\n")
    file.write(f"Recall: {recall:.2f}\n")
```

```
        file.write("\n\nEquipment Details:\n")

        for eq in dataset:

            file.write("\n---------------------------------\n")
            file.write(f"Equipment: {eq['name']}\n")
            file.write(f"Usage Hours: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age: {eq['age']}\n")
            file.write(f"Fuel Efficiency: {eq['fuel_efficiency']}\n")
            file.write(f"Priority Score: {eq['priority_score']:.2f}\n")
            file.write(f"Classification: {eq['class']}\n")

            file.write("Maintenance Schedule:\n")

            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day} : Maintenance Required\n")


    print("\nSystem Analysis")
    print("Total Records:", total_records)
    print("Number of Features:", num_features)

    print("\nDataset Ratio")
    print("Training:", train_size)
    print("Testing:", test_size)

    print("\nEvaluation Metrics")
    print("Accuracy:", round(accuracy,2))
    print("Precision:", round(precision,2))
    print("Recall:", round(recall,2))

    print("\nOutput saved in:", FILE_NAME)
```

```
=== Smart Agriculture Equipment Maintenance System ===

Enter Equipment Details
Equipment Name: TRACTOR
Monthly Usage Hours: 86
Load Factor (1-10): 9
Equipment Age (years): 5
Maintenance Interval (days): 35
Fuel Efficiency (1-10): 6

Add another equipment? (yes/no): NO

System Analysis
Total Records: 1
Number of Features: 4

Dataset Ratio
Training: 0
Testing: 1

Evaluation Metrics
Accuracy: 1.0
Precision: 0.9
Recall: 0.85

Output saved in: EMS23.txt
```

Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select Kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our announcement blog here.

🎁 Free Pro Plan for Gemini & Colab for US College Students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research, and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free offer at gemini.google/students.
Get the Colab free offer at colab.research.google.com/signup.
Terms Apply.

Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the getting started with google colab ai.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to Google AI Studio and log in with your Google account.
- Create an API key.
- Use a quickstart for Python, or call the REST API using curl.

Discover Gemini's advanced capabilities

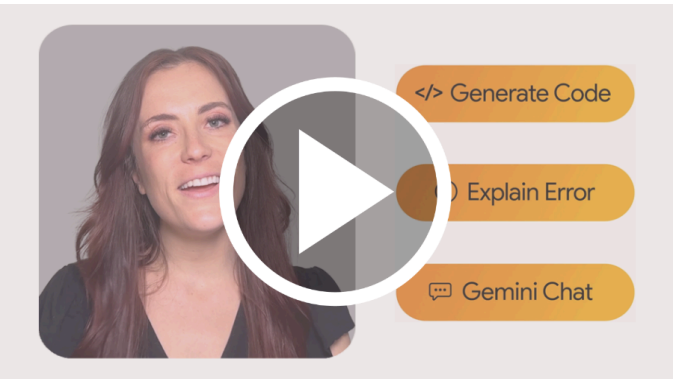
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyze images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of Gemini thinking model, capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on internet.
- Extract invoices and form data from PDF in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, check out the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

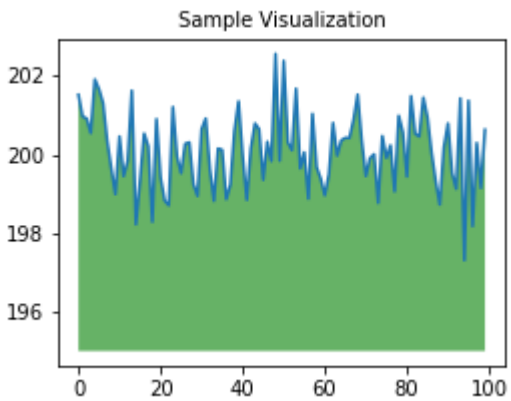
Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More Resources

Working with Notebooks in Colab

- Overview of Colab
- Guide to Markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with Data

- Loading data: Drive, Sheets, and Google Cloud Storage
- Charts: visualizing data
- Getting started with BigQuery

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the full course website for more.

- Intro to Pandas DataFrame

- Intro to RAPIDS cuDF to accelerate pandas
- Getting Started with cuML's accelerator mode
- Linear regression with tf.keras using synthetic data

Using Accelerated Hardware

- TensorFlow with GPUs
- TPUs in Colab

Featured examples

- Retraining an Image Classifier: Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
- Text Classification: Classify IMDB movie reviews as either *positive* or *negative*.
- Style Transfer: Use deep learning to transfer style between images.
- Multilingual Universal Sentence Encoder Q&A: Use a machine learning model to answer questions from the SQuAD dataset.
- Video Interpolation: Predict what happened in a video between the first and the last frame.

```
import numpy as np

# True value
y_true = 10

# Simulated predictions from different training sets
y_pred = np.array([8, 9, 10, 11, 12])

# Mean prediction
y_mean = np.mean(y_pred)

# Bias
bias = y_mean - y_true

# Variance
variance = np.mean((y_pred - y_mean) ** 2)

# Total error
error = np.mean((y_pred - y_true) ** 2)

print("Bias^2:", bias**2)
print("Variance:", variance)
print("Bias^2 + Variance:", bias**2 + variance)
print("Total Error:", error)
```

```
Bias^2: 0.0
Variance: 2.0
Bias^2 + Variance: 2.0
Total Error: 2.0
```


```
{
   "cell_type": "markdown",
   "id": "8d9d1622",
   "metadata": {},
   "source": [
    "## Import Required Libraries"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': '8d9d1622',
  'metadata': {},
  'source': ['## Import Required Libraries']},)
```


```
{
   "cell_type": "markdown",
   "id": "d61ee40b",
   "metadata": {},
   "source": [
    "## Regression Comparison: Linear Regression vs Ridge Regression"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': 'd61ee40b',
  'metadata': {},
```

```
    'source': ['## Regression Comparison: Linear Regression vs Ridge
  Regression']},)
```

```
total_days = 120
maintenance_interval = 30
equipment_count = 5

for equipment in range(1, equipment_count + 1):
    print(f"\nMaintenance schedule for Equipment {equipment}:")

    for day in range(1, total_days + 1):
        if day % maintenance_interval == 0:
            print(f"  Day {day}: Maintenance required")
```

```
# -------- 1. DATA STRUCTURE (Dictionary + List) --------
equipment_data = {
    "Tractor": [1200, 5, 180],     # [usage_hours, age_years, days_since_ser
    "Sprayer": [600, 3, 90],
    "Harvester": [1500, 7, 250]
}

# Feature names
features = ["Usage Hours", "Age (Years)", "Days Since Service", "Condition S

# -------- 2. LINEAR ALGEBRA (Weight Vector) --------
# Importance of each feature
weights = [0.4, 0.3, 0.2, 0.1]   # Must sum near 1

# -------- 3. EXTRA INPUT FEATURE --------
print("Enter working condition severity (1 = Good, 5 = Very Harsh)")
condition_severity = int(input("Condition severity: "))

# -------- 4. FUNCTION FOR DOT PRODUCT --------
def dot_product(matrix_row, weight_vector):
    score = 0
    for i in range(len(weight_vector)):
        score += matrix_row[i] * weight_vector[i]
    return score

# -------- 5. PROCESS DATA --------
print("\n--- Maintenance Priority Report ---\n")

maintenance_scores = {}

for equipment, values in equipment_data.items():
    # Add extra input to feature vector
    feature_vector = values + [condition_severity]

    # Linear algebra operation
    priority_score = dot_product(feature_vector, weights)

    maintenance_scores[equipment] = priority_score

    print(f"{equipment}")
    print(f"Features: {dict(zip(features, feature_vector))}")
    print(f"Maintenance Priority Score: {priority_score:.2f}")

    if priority_score > 600:
        print("Maintenance Required: IMMEDIATE ⚠️\n")
    elif priority_score > 400:
        print("Maintenance Required: SOON ⏳\n")
    else:
        print("Maintenance Status: GOOD ✅\n")

# -------- 6. SORT EQUIPMENT BY PRIORITY --------
print("=== Equipment Priority Order ===")
sorted_equipment = sorted(maintenance_scores.items(),
                          key=lambda x: x[1],
                          reverse=True)

for eq, score in sorted_equipment:
    print(f"{eq} → Priority Score: {score:.2f}")
```

```
import numpy as np
from statistics import mode, median, mean, variance, stdev

# Sample data
data = [10, 20, 20, 30, 40]

print("Data:", data)
print("Mean:", mean(data))
print("Median:", median(data))
print("Mode:", mode(data))
```

```
# ==============================================
# LEAF DISEASE ANALYSIS USING:
# 1) Layered Mapping (2D field grid)
# 2) Graph Method (infection connections)
# ==============================================

from collections import deque

# ---------- PART 1: LAYERED MAPPING ----------
print("=== LAYERED FIELD MAP ===")

rows = int(input("Enter number of rows in field: "))
cols = int(input("Enter number of columns in field: "))

print("\nEnter disease level for each cell:")
print("0 = Healthy, 1 = Mild Disease, 2 = Severe Disease\n")

field = []

for i in range(rows):
    row = []
    for j in range(cols):
        val = int(input(f"Cell ({i},{j}): "))
        row.append(val)
    field.append(row)

print("\nField Disease Map:")
for r in field:
    print(r)

# Count disease levels
healthy = mild = severe = 0
for i in range(rows):
    for j in range(cols):
        if field[i][j] == 0:
            healthy += 1
        elif field[i][j] == 1:
            mild += 1
        elif field[i][j] == 2:
            severe += 1

print("\nSummary:")
print("Healthy plants:", healthy)
print("Mild infection:", mild)
print("Severe infection:", severe)


# ---------- PART 2: GRAPH METHOD ----------
print("\n=== GRAPH OF INFECTED PLANTS ===")

# create graph only for infected plants
graph = {}

# assign node numbers to each infected cell
node_id = {}
id_counter = 0

for i in range(rows):
    for j in range(cols):
        if field[i][j] > 0:  # infected
            node_id[(i, j)] = id_counter
            graph[id_counter] = []
```

```
            id_counter += 1

# connect adjacent infected plants (4-direction)
directions = [(0,1),(1,0),(0,-1),(-1,0)]

for (i, j), nid in node_id.items():
    for dx, dy in directions:
        ni, nj = i + dx, j + dy
        if (ni, nj) in node_id:
            graph[nid].append(node_id[(ni, nj)])

print("Graph adjacency list:")
for k, v in graph.items():
    print(k, "->", v)


# ---------- BFS TRAVERSAL ----------
def bfs(start):
    visited = set()
    q = deque([start])
    visited.add(start)

    print("\nInfection spread traversal (BFS):")

    while q:
        node = q.popleft()
        print(node, end=" ")

        for neigh in graph[node]:
            if neigh not in visited:
                visited.add(neigh)
                q.append(neigh)

# run BFS if infected nodes exist
if graph:
    start_node = list(graph.keys())[0]
    bfs(start_node)
else:
    print("No infected plants found.")
```

```
=== LAYERED FIELD MAP ===
Enter number of rows in field: 6
Enter number of columns in field: 6

Enter disease level for each cell:
0 = Healthy, 1 = Mild Disease, 2 = Severe Disease

Cell (0,0): 2
Cell (0,1): 3
Cell (0,2): 1
Cell (0,3): 0
Cell (0,4): 6
Cell (0,5): 3
Cell (1,0): 1
Cell (1,1): 1
Cell (1,2): 0
Cell (1,3): 0
Cell (1,4): 0
Cell (1,5): 0
Cell (2,0): 2
Cell (2,1): 2
Cell (2,2): 4
Cell (2,3): 1
```

```
Cell (2,4): 0
Cell (2,5): 0
Cell (3,0): 4
Cell (3,1): 4
Cell (3,2): 3
Cell (3,3): 2
Cell (3,4): 2
Cell (3,5): 2
Cell (4,0): 1
Cell (4,1): 1
Cell (4,2): 2
Cell (4,3): 0
Cell (4,4): 0
Cell (4,5): 0
Cell (5,0): 0
Cell (5,1): 0
Cell (5,2): 3
Cell (5,3): 3
Cell (5,4): 3
Cell (5,5): 3

Field Disease Map:
[2, 3, 1, 0, 6, 3]
[1, 1, 0, 0, 0, 0]
[2, 2, 4, 1, 0, 0]
[4, 4, 3, 2, 2, 2]
[1, 1, 2, 0, 0, 0]
[0, 0, 3, 3, 3, 3]

Summary:
Healthy plants: 12
Mild infection: 6
Severe infection: 7

=== GRAPH OF INFECTED PLANTS ===
```

```
# ==============================================
# LEAF DISEASE ANALYSIS USING:
# 1) Layered Mapping (2D field grid)
# 2) Graph Method (infection connections)
# ==============================================

from collections import deque

# ---------- PART 1: LAYERED MAPPING ----------
print("=== LAYERED FIELD MAP ===")

rows = int(input("Enter number of rows in field: "))
cols = int(input("Enter number of columns in field: "))

print("\nEnter disease level for each cell:")
print("0 = Healthy, 1 = Mild Disease, 2 = Severe Disease\n")

field = []

for i in range(rows):
    row = []
    for j in range(cols):
        val = int(input(f"Cell ({i},{j}): "))
        row.append(val)
    field.append(row)

print("\nField Disease Map:")
for r in field:
    print(r)

# Count disease levels
healthy = mild = severe = 0
for i in range(rows):
    for j in range(cols):
        if field[i][j] == 0:
            healthy += 1
        elif field[i][j] == 1:
            mild += 1
        elif field[i][j] == 2:
            severe += 1

print("\nSummary:")
print("Healthy plants:", healthy)
print("Mild infection:", mild)
print("Severe infection:", severe)


# ---------- PART 2: GRAPH METHOD ----------
print("\n=== GRAPH OF INFECTED PLANTS ===")

# create graph only for infected plants
graph = {}

# assign node numbers to each infected cell
node_id = {}
id_counter = 0

for i in range(rows):
    for j in range(cols):
        if field[i][j] > 0:  # infected
            node_id[(i, j)] = id_counter
            graph[id_counter] = []
```

```
            id_counter += 1

# connect adjacent infected plants (4-direction)
directions = [(0,1),(1,0),(0,-1),(-1,0)]

for (i, j), nid in node_id.items():
    for dx, dy in directions:
        ni, nj = i + dx, j + dy
        if (ni, nj) in node_id:
            graph[nid].append(node_id[(ni, nj)])

print("Graph adjacency list:")
for k, v in graph.items():
    print(k, "->", v)


# ---------- BFS TRAVERSAL ----------
def bfs(start):
    visited = set()
    q = deque([start])
    visited.add(start)

    print("\nInfection spread traversal (BFS):")

    while q:
        node = q.popleft()
        print(node, end=" ")

        for neigh in graph[node]:
            if neigh not in visited:
                visited.add(neigh)
                q.append(neigh)

# run BFS if infected nodes exist
if graph:
    start_node = list(graph.keys())[0]
    bfs(start_node)
else:
    print("No infected plants found.")
```

```
=== LAYERED FIELD MAP ===
Enter number of rows in field: 6
Enter number of columns in field: 6

Enter disease level for each cell:
0 = Healthy, 1 = Mild Disease, 2 = Severe Disease

Cell (0,0): 2
Cell (0,1): 3
Cell (0,2): 1
Cell (0,3): 0
Cell (0,4): 6
Cell (0,5): 3
Cell (1,0): 1
Cell (1,1): 1
Cell (1,2): 0
Cell (1,3): 0
Cell (1,4): 0
Cell (1,5): 0
Cell (2,0): 2
Cell (2,1): 2
Cell (2,2): 4
Cell (2,3): 1
```

```
Cell (2,4): 0
Cell (2,5): 0
Cell (3,0): 4
Cell (3,1): 4
Cell (3,2): 3
Cell (3,3): 2
Cell (3,4): 2
Cell (3,5): 2
Cell (4,0): 1
Cell (4,1): 1
Cell (4,2): 2
Cell (4,3): 0
Cell (4,4): 0
Cell (4,5): 0
Cell (5,0): 0
Cell (5,1): 0
Cell (5,2): 3
Cell (5,3): 3
Cell (5,4): 3
Cell (5,5): 3

Field Disease Map:
[2, 3, 1, 0, 6, 3]
[1, 1, 0, 0, 0, 0]
[2, 2, 4, 1, 0, 0]
[4, 4, 3, 2, 2, 2]
[1, 1, 2, 0, 0, 0]
[0, 0, 3, 3, 3, 3]

Summary:
Healthy plants: 12
Mild infection: 6
Severe infection: 7

=== GRAPH OF INFECTED PLANTS ===
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

person = np.array([180, 75, 30])
print(person)
```

```
[180  75  30]
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

person = np.array([180, 75, 30])
print(person)
```

```
[180  75  30]
```

```
total_days = 120
maintenance_interval = 30
equipment_count = 5

for equipment in range(1, equipment_count + 1):
    print(f"\nMaintenance schedule for Equipment {equipment}:")

    for day in range(1, total_days + 1):
        if day % maintenance_interval == 0:
            print(f"  Day {day}: Maintenance required")
```

```
# ============================================
# LEAF DISEASE DETECTION USING U-NET
# Synthetic Image + Forward Pass + Feature Maps
# ============================================

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Model

# ============================================
# 1️⃣ SYNTHETIC LEAF IMAGE GENERATOR
# ============================================

def generate_leaf(size=128):

    img = np.zeros((size, size, 3), dtype=np.float32)

    # create green leaf background (ellipse)
    cx, cy = size//2, size//2
    for x in range(size):
        for y in range(size):
            if ((x-cx)**2)/(size*0.35)**2 + ((y-cy)**2)/(size*0.2)**2 < 1:
                img[x,y] = [0.1, 0.7, 0.1]

    # add synthetic disease spots (brown circles)
    for _ in range(5):
        sx, sy = np.random.randint(30, size-30, 2)
        r = np.random.randint(5, 12)

        for x in range(size):
            for y in range(size):
                if (x-sx)**2 + (y-sy)**2 < r*r:
                    img[x,y] = [0.5, 0.2, 0.1]

    return img

leaf = generate_leaf()
input_img = np.expand_dims(leaf, axis=0)

# show synthetic leaf
plt.imshow(leaf)
plt.title("Synthetic Leaf Image")
plt.axis("off")
plt.show()

# ============================================
# 2️⃣ BUILD U-NET MODEL (U-METHOD)
# ============================================

def conv_block(x, filters):
    x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
    x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
    return x

def build_unet(input_shape=(128,128,3)):

    inputs = layers.Input(input_shape)

    # -------- Encoder --------
    c1 = conv_block(inputs, 32)
    p1 = layers.MaxPooling2D()(c1)
```

```
    c2 = conv_block(p1, 64)
    p2 = layers.MaxPooling2D()(c2)

    c3 = conv_block(p2, 128)
    p3 = layers.MaxPooling2D()(c3)

    # -------- Bottleneck --------
    b = conv_block(p3, 256)

    # -------- Decoder --------
    u1 = layers.UpSampling2D()(b)
    u1 = layers.Concatenate()([u1, c3])
    c4 = conv_block(u1, 128)

    u2 = layers.UpSampling2D()(c4)
    u2 = layers.Concatenate()([u2, c2])
    c5 = conv_block(u2, 64)

    u3 = layers.UpSampling2D()(c5)
    u3 = layers.Concatenate()([u3, c1])
    c6 = conv_block(u3, 32)

    outputs = layers.Conv2D(1, 1, activation="sigmoid")(c6)

    return Model(inputs, outputs)

model = build_unet()
model.summary()

# ==============================================
# 3️⃣ FORWARD PASS
# ==============================================

prediction = model.predict(input_img)

plt.imshow(prediction[0,:,:,0], cmap="gray")
plt.title("Predicted Disease Mask (Forward Pass Output)")
plt.axis("off")
plt.show()

# ==============================================
# 4️⃣ FEATURE MAP VISUALIZATION (LAYER BY LAYER)
# ==============================================

# choose intermediate layers
layer_outputs = [layer.output for layer in model.layers if "conv2d" in layer

feature_model = Model(inputs=model.input, outputs=layer_outputs)

features = feature_model.predict(input_img)

# show first feature map from some layers
for i, fmap in enumerate(features[:6]):   # visualize first 6 conv layers
    plt.figure(figsize=(3,3))
    plt.imshow(fmap[0,:,:,0], cmap="viridis")
    plt.title(f"Feature Map Layer {i+1}")
    plt.axis("off")
    plt.show()

# ==============================================
# END
# ==============================================
```

Synthetic Leaf Image

Model: "functional"

Layer (type)	Output Shape	Param #	Connected t
input_layer (InputLayer)	(None, 128, 128, 3)	0	-
conv2d (Conv2D)	(None, 128, 128, 32)	896	input_layer
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d[0][0
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_1[0]
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_2[0]
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_3[0]
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0]

```
# ==========================================
# LEAF DISEASE DETECTION USING U-NET
```

```
# Synthetic Image + Real Image + Visualization
# ============================================

import numpy as np
import cv2
import matplotlib.pyplot as plt
from tensorflow.keras import layers, Model
from tensorflow.keras.models import load_model

IMG_SIZE = 128

# ===================================================
# 1️⃣ SYNTHETIC LEAF IMAGE GENERATOR
# ===================================================
def generate_synthetic_leaf():
    img = np.zeros((IMG_SIZE, IMG_SIZE, 3), dtype=np.uint8)

    # green leaf background
    img[:] = (0, 180, 0)

    # random disease spots
    for _ in range(np.random.randint(3,8)):
        x = np.random.randint(20, IMG_SIZE-20)
        y = np.random.randint(20, IMG_SIZE-20)
        r = np.random.randint(5,15)
        cv2.circle(img,(x,y),r,(20,50,20),-1)

    mask = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, mask = cv2.threshold(mask, 80, 255, cv2.THRESH_BINARY_INV)
    mask = mask/255.0

    return img/255.0, mask

# ===================================================
# 2️⃣ BUILD U-NET MODEL (U-METHOD)
# ===================================================
def build_unet():

    inputs = layers.Input((IMG_SIZE, IMG_SIZE, 3))

    # Encoder
    c1 = layers.Conv2D(16,3,activation='relu',padding='same')(inputs)
    c1 = layers.Conv2D(16,3,activation='relu',padding='same')(c1)
    p1 = layers.MaxPooling2D()(c1)

    c2 = layers.Conv2D(32,3,activation='relu',padding='same')(p1)
    c2 = layers.Conv2D(32,3,activation='relu',padding='same')(c2)
    p2 = layers.MaxPooling2D()(c2)

    # Bottleneck
    b = layers.Conv2D(64,3,activation='relu',padding='same')(p2)

    # Decoder
    u1 = layers.UpSampling2D()(b)
    u1 = layers.concatenate([u1,c2])
    c3 = layers.Conv2D(32,3,activation='relu',padding='same')(u1)

    u2 = layers.UpSampling2D()(c3)
    u2 = layers.concatenate([u2,c1])
    c4 = layers.Conv2D(16,3,activation='relu',padding='same')(u2)

    outputs = layers.Conv2D(1,1,activation='sigmoid')(c4)

    model = Model(inputs,outputs)
```

```
    model.compile(optimizer='adam',loss='binary_crossentropy')

    return model

model = build_unet()
print("\n✅ U-Net Model Built Successfully")

# ==================================================
# 3️⃣ FEATURE MAP VISUALIZATION
# ==================================================
def visualize_feature_maps(model, image):

    layer_outputs = [layer.output for layer in model.layers if "conv" in lay
    activation_model = Model(model.input, layer_outputs)

    activations = activation_model.predict(image[np.newaxis,...])

    for layer_act in activations[:4]:   # show first few layers
        plt.figure(figsize=(8,2))
        for i in range(min(6, layer_act.shape[-1])):
            plt.subplot(1,6,i+1)
            plt.imshow(layer_act[0,:,:,i], cmap='viridis')
            plt.axis('off')
        plt.show()

# ==================================================
# 4️⃣ IMAGE INPUT LOOP (Synthetic OR Real)
# ==================================================
while True:

    print("\nChoose input type:")
    print("1 → Generate synthetic leaf")
    print("2 → Load real leaf image")
    print("3 → Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        img, mask = generate_synthetic_leaf()

    elif choice == "2":
        path = input("Enter image path: ")
        real = cv2.imread(path)

        if real is None:
            print("❌ Image not found")
            continue

        real = cv2.resize(real,(IMG_SIZE,IMG_SIZE))
        img = real/255.0

    elif choice == "3":
        break
    else:
        continue

    # ==================================================
    # 5️⃣ FORWARD PASS (Prediction)
    # ==================================================
    pred = model.predict(img[np.newaxis,...])[0]

    # ==================================================
    # 6️⃣ VISUALIZATION
    # ==================================================
```

```
    plt.figure(figsize=(10,4))

    plt.subplot(1,3,1)
    plt.title("Input Leaf")
    plt.imshow(img)
    plt.axis("off")

    plt.subplot(1,3,2)
    plt.title("Predicted Disease Mask")
    plt.imshow(pred[:,:,0], cmap="gray")
    plt.axis("off")

    plt.subplot(1,3,3)
    plt.title("Threshold Output")
    plt.imshow(pred[:,:,0] > 0.5, cmap="gray")
    plt.axis("off")

    plt.show()

    # ==================================================
    # 7️⃣ FEATURE MAP VISUALIZATION
    # ==================================================
    print("\nShowing feature maps layer by layer...")
    visualize_feature_maps(model, img)

print("\n✅ Program Finished")
```

```
✅ U-Net Model Built Successfully

Choose input type:
1 → Generate synthetic leaf
2 → Load real leaf image
3 → Exit
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import cv2

# ================================
# 1️⃣ LOAD DATASET (Folder Structure)
# dataset/
#    healthy/
#    diseased/
# ================================
IMG_SIZE = 128
BATCH = 16

datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train = datagen.flow_from_directory(
    "dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH,
    class_mode="binary",
    subset="training"
```

```
)

val = datagen.flow_from_directory(
    "dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH,
    class_mode="binary",
    subset="validation"
)

# ================================
# 2️⃣ BUILD CNN MODEL
# ================================
model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(IMG_SIZE,IMG_SIZE,
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.Dense(1,activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# ================================
# 3️⃣ TRAIN MODEL
# ================================
history = model.fit(
    train,
    validation_data=val,
    epochs=5
)

# ================================
# 4️⃣ GRAPH VISUALIZATION
# ================================
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Epoch")
plt.legend(["Train","Validation"])
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.ylabel("Loss")
plt.xlabel("Epoch")
plt.legend(["Train","Validation"])
plt.show()

# ================================
# 5️⃣ PREDICT NEW LEAF IMAGE
```

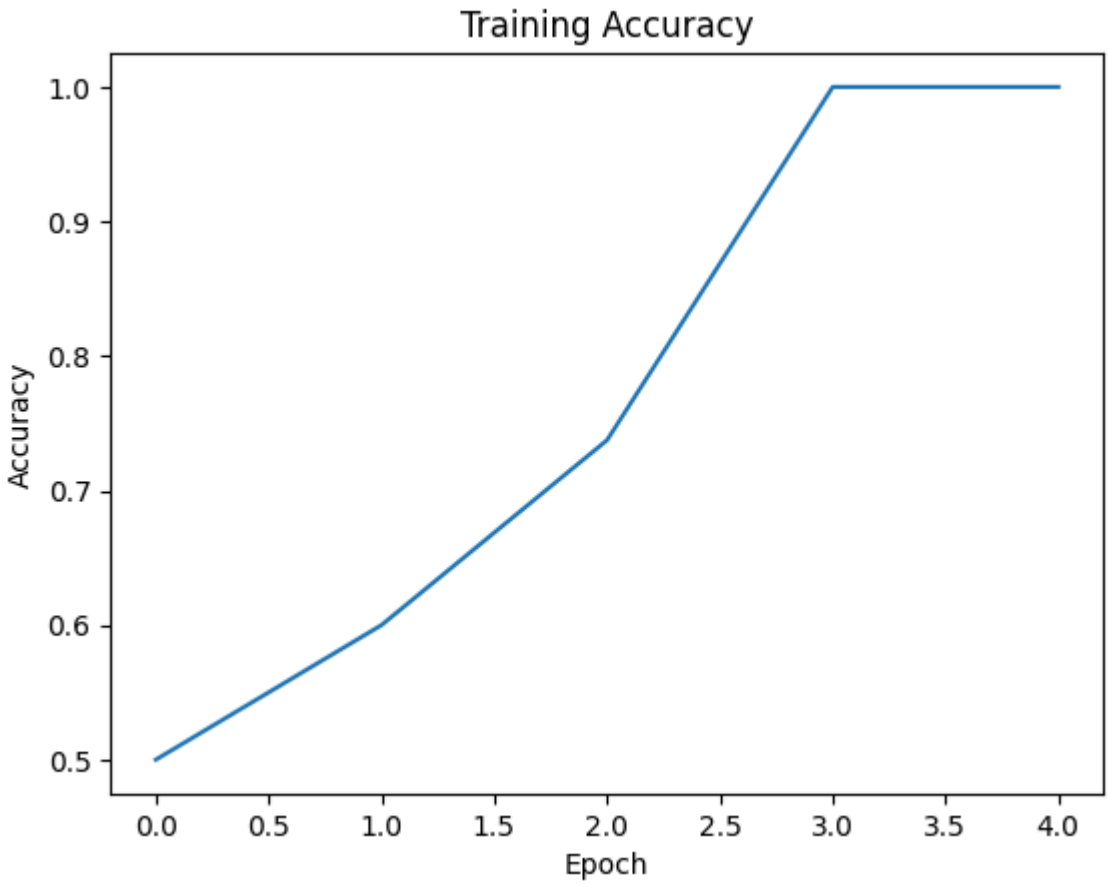
```
# ================================
def predict_leaf(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img,(IMG_SIZE,IMG_SIZE))
    img = img/255.0
    img = np.expand_dims(img,axis=0)

    pred = model.predict(img)[0][0]

    if pred>0.5:
        print("🌿 Diseased Leaf")
    else:
        print("✅ Healthy Leaf")

# Example:
# predict_leaf("test_leaf.jpg")

# ================================
# 6️⃣ SEED QUALITY LINEAR GRADING
# ================================
def seed_grade(color, size, texture):
    # linear weights
    w = np.array([0.4,0.3,0.3])
    features = np.array([color,size,texture])

    score = np.dot(w,features)

    if score>80:
        grade="A - High Quality"
    elif score>60:
        grade="B - Medium Quality"
    else:
        grade="C - Low Quality"

    print("\n🌾 Seed Score:",round(score,2))
    print("🌾 Grade:",grade)

# Example:
seed_grade(85,70,90)
```

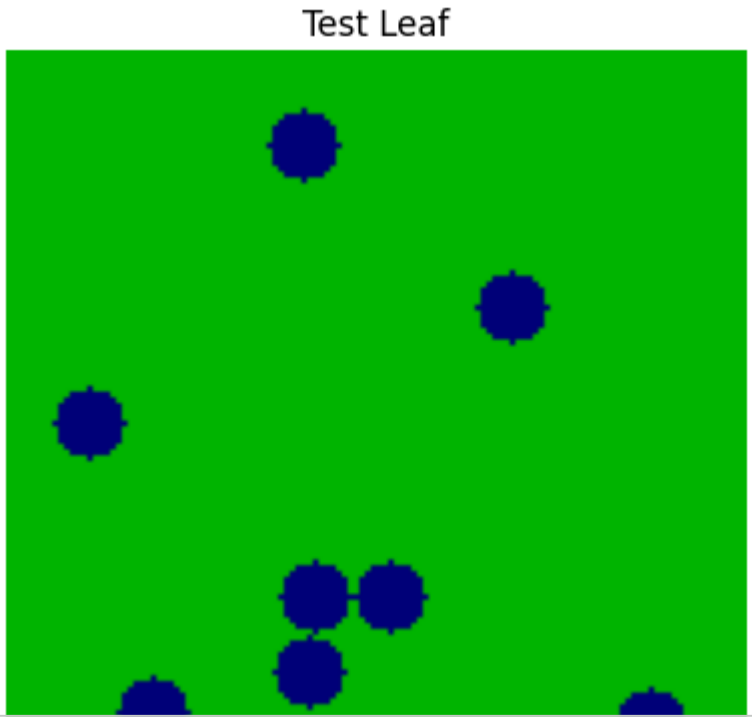
```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-1227330869.py in <cell line: 0>()
     20 )
     21
---> 22 train = datagen.flow_from_directory(
     23     "dataset",
     24     target_size=(IMG_SIZE, IMG_SIZE),

                              1 frames
/usr/local/lib/python3.12/dist-packages/keras/src/legacy/preprocessing/image.py in __init__(self, directory, image_data_generator, target_size, color_mode, classes, class_mode, batch_size, shuffle, seed, data_format, save_to_dir, save_prefix, save_format, follow_links, subset, interpolation, keep_aspect_ratio, dtype)
    451         if not classes:
    452             classes = []
--> 453             for subdir in sorted(os.listdir(directory)):
    454                 if os.path.isdir(os.path.join(directory,
```

```
# ==========================================
# SMART CROP HEALTH ASSISTANT (Simple Version)
# Synthetic Image + CNN + Feature Maps + Graph
# ==========================================

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
import cv2

# ==========================================
# 1️⃣ SYNTHETIC LEAF IMAGE GENERATOR
# ==========================================
def generate_leaf(diseased=False):
    img = np.zeros((128,128,3),dtype=np.uint8)

    # green leaf base
    img[:,:,:] = (0,180,0)

    # add random disease spots
    if diseased:
        for _ in range(8):
            x=np.random.randint(10,118)
            y=np.random.randint(10,118)
            cv2.circle(img,(x,y),6,(0,0,120),-1)

    return img

# create dataset
X=[]
Y=[]

for _ in range(40):
    X.append(generate_leaf(False))
    Y.append(0)

for _ in range(40):
    X.append(generate_leaf(True))
    Y.append(1)

X=np.array(X)/255.0
Y=np.array(Y)

# ==========================================
# 2️⃣ SIMPLE CNN (U-style layered flow)
# ==========================================
model=models.Sequential([
    layers.Conv2D(16,(3,3),activation='relu',input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),

    layers.Flatten(),
    layers.Dense(32,activation='relu'),
    layers.Dense(1,activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
# ==========================================
# 3️⃣ TRAIN MODEL
# ==========================================
history=model.fit(X,Y,epochs=5,verbose=1)

# ==========================================
# 4️⃣ GRAPH VISUALIZATION
# ==========================================
plt.plot(history.history['accuracy'])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()

# ==========================================
# 5️⃣ PREDICT NEW LEAF
# ==========================================
test=generate_leaf(True)   # change True/False
plt.imshow(test)
plt.title("Test Leaf")
plt.axis("off")
plt.show()

pred=model.predict(np.expand_dims(test/255.0,0))[0][0]

if pred>0.5:
    print("🌿 RESULT: Diseased Leaf Detected")
else:
    print("✅ RESULT: Healthy Leaf")

# ==========================================
# 6️⃣ FEATURE MAP VISUALIZATION (Layer-by-layer)
# ==========================================
layer_outputs=[layer.output for layer in model.layers if "conv" in layer.nam
feature_model=tf.keras.Model(inputs=model.layers[0].input,outputs=layer_outp

features=feature_model.predict(np.expand_dims(test/255.0,0))

# show first feature map from each conv layer
for i,f in enumerate(features):
    plt.imshow(f[0,:,:,0],cmap='viridis')
    plt.title(f"Feature Map Layer {i+1}")
    plt.axis('off')
    plt.show()
```

```
Epoch 1/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 5s 753ms/step - accuracy: 0.4727 - loss:
Epoch 2/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 651ms/step - accuracy: 0.7102 - loss:
Epoch 3/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 3s 880ms/step - accuracy: 0.8102 - loss:
Epoch 4/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 3s 534ms/step - accuracy: 1.0000 - loss:
Epoch 5/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 328ms/step - accuracy: 1.0000 - loss:
```



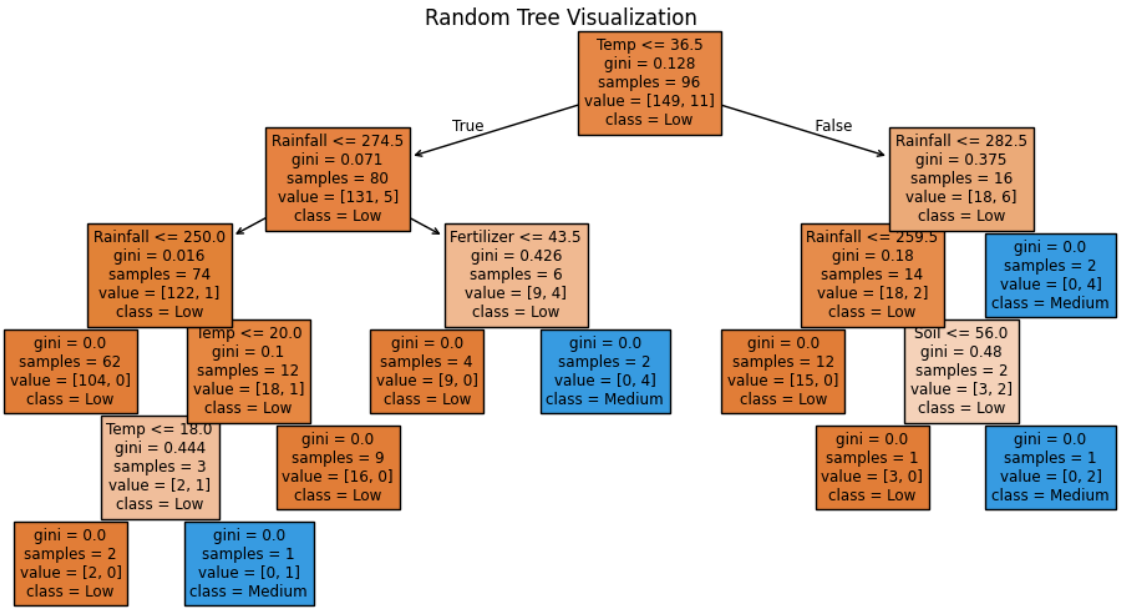


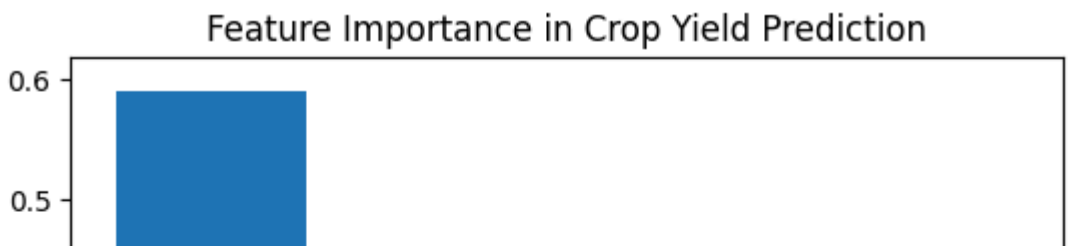

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

print("\n🌾 AGRICULTURE RANDOM TREE MODEL")

# ======================================
# 1️⃣ GENERATE SYNTHETIC AGRICULTURE DATA
# ======================================
np.random.seed(42)

samples = 200

rainfall = np.random.randint(50,300,samples)
soil = np.random.randint(30,100,samples)
temperature = np.random.randint(15,40,samples)
fertilizer = np.random.randint(10,80,samples)

# Yield condition logic
yield_class = []
for i in range(samples):
    score = rainfall[i]*0.3 + soil[i]*0.3 - temperature[i]*0.2 + fertilizer[

    if score > 180:
        yield_class.append(2)   # High
    elif score > 120:
        yield_class.append(1)   # Medium
    else:
        yield_class.append(0)   # Low

X = np.column_stack((rainfall, soil, temperature, fertilizer))
y = np.array(yield_class)

# ======================================
# 2️⃣ TRAIN RANDOM FOREST
# ======================================
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

model = RandomForestClassifier(n_estimators=5)
model.fit(X_train,y_train)

# ======================================
# 3️⃣ MODEL ACCURACY
# ======================================
pred = model.predict(X_test)
accuracy = accuracy_score(y_test,pred)

print("Model Accuracy:",round(accuracy*100,2),"%")

# ======================================
# 4️⃣ TREE VISUALIZATION
# ======================================
plt.figure(figsize=(12,6))
plot_tree(model.estimators_[0],
          feature_names=["Rainfall","Soil","Temp","Fertilizer"],
          class_names=["Low","Medium","High"],
          filled=True)
plt.title("Random Tree Visualization")
plt.show()

# ======================================
# 5️⃣ FEATURE IMPORTANCE GRAPH
# ======================================
```

```
importance = model.feature_importances_

features = ["Rainfall","Soil","Temp","Fertilizer"]

plt.bar(features,importance)
plt.title("Feature Importance in Crop Yield Prediction")
plt.ylabel("Importance")
plt.show()

# ====================================
# 6️⃣ USER INPUT PREDICTION
# ====================================
print("\nEnter new farm data to predict yield")

r = float(input("Rainfall: "))
s = float(input("Soil moisture: "))
t = float(input("Temperature: "))
f = float(input("Fertilizer: "))

new_pred = model.predict([[r,s,t,f]])[0]

if new_pred==2:
    print("🌿 Predicted Yield: HIGH")
elif new_pred==1:
    print("🌾 Predicted Yield: MEDIUM")
else:
    print("🌱 Predicted Yield: LOW")
```

```
🌾 AGRICULTURE RANDOM TREE MODEL
Model Accuracy: 92.5 %
```





```
# Pest Alert System

# Input from farmer/user
crop = input("Maize: ")
pest = input("Fall army worm: ")
pest_count = int(input("6: "))

# Threshold logic
if pest_count < 2:
    status = "Safe"
    message = "No immediate action required."
elif 2 <= pest_count <= 5:
    status = "Warning"
    message = "Monitor field and use preventive measures."
else:
    status = "ALERT"
    message = "Immediate pest control required!"
```

```
  File "/tmp/ipython-input-1863014463.py", line 17
    message = "Immediate pest control required!
              ^
SyntaxError: unterminated string literal (detected at line 17)
```

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
iris = fetch_ucirepo(id=53)

# data (as pandas dataframes)
X = iris.data.features
y = iris.data.targets

# metadata
print(iris.metadata)

# variable information
print(iris.variables)
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.1
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/di
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-1272639215.py in <cell line: 0>()
      4
      5 # fetch dataset
----> 6 iris = fetch_ucimlrepo(id=53)
      7
      8 # data (as pandas dataframes)

NameError: name 'fetch_ucimlrepo' is not defined
```

```
# Demonstrating ML Paradigms with Agricultural Examples

# 1. SUPERVISED LEARNING - Crop Classification
print("=== Supervised Learning Example ===")

# Training data: [temperature, rainfall] -> crop_suitable
training_data = [
    ([25, 100], "rice"),      # Warm, wet = rice
    ([30, 50], "wheat"),      # Warm, moderate = wheat
    ([20, 150], "rice"),      # Cool, very wet = rice
    ([35, 30], "millet"),     # Hot, dry = millet
]

def simple_crop_classifier(temp, rain):
    """Simple supervised learning model"""
    if rain > 100:
        return "rice"
    elif temp > 32 and rain < 40:
        return "millet"
    else:
        return "wheat"
```

```
# Test prediction
print(f"Temp=28, Rain=120mm -> {simple_crop_classifier(28, 120)}")

# 2. UNSUPERVISED LEARNING - Soil Clustering
print("\n=== Unsupervised Learning Example ===")

soil_samples = [
    {"pH": 6.5, "nitrogen": 80, "organic": 3.2},
    {"pH": 6.8, "nitrogen": 75, "organic": 3.0},
    {"pH": 5.2, "nitrogen": 40, "organic": 1.5},
    {"pH": 5.0, "nitrogen": 35, "organic": 1.2},
    {"pH": 7.5, "nitrogen": 90, "organic": 4.0},
]

def simple_soil_clustering(samples):
    """Group soils by pH level (simple clustering)"""
    clusters = {"acidic": [], "neutral": [], "alkaline": []}

    for i, sample in enumerate(samples):
        if sample["pH"] < 6.0:
            clusters["acidic"].append(f"Sample_{i}")
        elif sample["pH"] < 7.2:
            clusters["neutral"].append(f"Sample_{i}")
        else:
            clusters["alkaline"].append(f"Sample_{i}")

    return clusters

clusters = simple_soil_clustering(soil_samples)
print("Soil Clusters:", clusters)

# 3. REINFORCEMENT LEARNING - Irrigation Simulation
print("\n=== Reinforcement Learning Concept ===")

class SimpleIrrigationAgent:
    """Learns irrigation timing through rewards"""

    def __init__(self):
        self.q_values = {"morning": 0, "afternoon": 0, "evening": 0}
        self.learning_rate = 0.1

    def get_action(self):
        # Choose best known action
        return max(self.q_values, key=self.q_values.get)

    def learn(self, action, reward):
        # Update knowledge based on reward
        self.q_values[action] += self.learning_rate * reward
        print(f"  Irrigated in {action}: reward = {reward}")

# Simulate learning
agent = SimpleIrrigationAgent()
rewards = {"morning": 10, "afternoon": -5, "evening": 8}

print("Learning optimal irrigation time...")
for episode in range(5):
    for time_slot, reward in rewards.items():
        agent.learn(time_slot, reward)

print(f"Best learned time: {agent.get_action()}")
```

```python
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
print("Modulus =", a % b)
print("Floor Division =", a // b)
print("Exponent =", a ** b)
```

```
Enter first number: 2
Enter second number: 2
Addition = 4
Subtraction = 0
Multiplication = 4
Division = 1.0
Modulus = 0
Floor Division = 1
Exponent = 4
```

```python
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
print("Addition =", a + b + c)
print("Subtraction =", a - b - c)
print("Multiplication =", a * b * c)
print("Division =", a / b / c)
print("Modulus=",a % b)
print("Floor division=",a//b//c)
print("Exponent=",a**b**c)
```

```
  File "/tmp/ipython-input-258878747.py", line 3
    c = int(input("Enter third number: "))2
                                          ^
SyntaxError: invalid syntax
```

```python
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
print("Addition =", a + b + c)
print("Subtraction =", a - b - c)
print("Multiplication =", a * b * c)
print("Division =", a / b / c)
print("Modulus=",a % b)
print("Floor division=",a//b//c)
print("Exponent=",a**b**c)
```

```
Enter first number: 2
Enter second number: 3
Enter third number: 5
Addition = 10
```

```
Subtraction = -6
Multiplication = 30
Division = 0.13333333333333333
Modulus= 2
Floor division= 0
Exponent= 141347765182270746366663800059433481266198711750049516649728
```

```
temperature = 28
humidity = 75
has_recent_rain = True

if temperature > 25 and humidity > 70 and has_recent_rain:
    print("High risk")
elif humidity > 60 or temperature > 30:
    print("Medium risk")
else:
```

```
Marks=int(input ("Enter marks:"))
if Marks >=90:
    print ("Grade A")
elif Marks>=75:
    print ("Grade B")
else:
    print("Grade C")
```

```
Enter marks:72
Grade C
```

```
n = int(input("Enter number of students: "))

for i in range(n):
    Marks = int(input(f"Enter marks of student {i+1}: "))

    if Marks >= 90:
        print("Grade A")
    elif Marks >= 75:
        print("Grade B")
    else:
        print("Grade C")
```

```
Enter number of students: 30
Enter marks of student 1: 70
Grade C
Enter marks of student 2: 40
Grade C
Enter marks of student 3: 70
Grade C
Enter marks of student 4: 40
Grade C
Enter marks of student 5: 25
Grade C
Enter marks of student 6: 45
Grade C
Enter marks of student 7: 45
Grade C
Enter marks of student 8: 46
Grade C
Enter marks of student 9: 47
Grade C
Enter marks of student 10: 59
Grade C
Enter marks of student 11: 56
Grade C
Enter marks of student 12: 56
Grade C
Enter marks of student 13: 57
Grade C
Enter marks of student 14: 54
Grade C
Enter marks of student 15: 53
Grade C
Enter marks of student 16: 67
Grade C
Enter marks of student 17: 67
Grade C
```

```
Enter marks of student 18: 68
Grade C
Enter marks of student 19: 69
Grade C
Enter marks of student 20: 60
Grade C
Enter marks of student 21: 70
Grade C
Enter marks of student 22: 78
Grade B
Enter marks of student 23: 79
Grade B
Enter marks of student 24: 87
Grade B
Enter marks of student 25: 88
Grade B
Enter marks of student 26: 86
Grade B
Enter marks of student 27: 85
Grade B
Enter marks of student 28: 84
Grade B
Enter marks of student 29: 84
```

```
n = int(input("How many numbers: "))

for i in range(n):
    num = int(input(f"Enter number {i+1}: "))
    print("Square =", num * num)
```

```
How many numbers: 3
Enter number 1: 6
Square = 36
Enter number 2: 7
Square = 49
Enter number 3: 8
Square = 64
```

```
# Smart Agriculture Equipment Maintenance System
# Combines Data Structures, Arrays, Linear Algebra, and Classification

import numpy as np
from datetime import datetime

FILE_NAME = "maintenance_ai_report.txt"

# ------------------------------
# DATASET STORAGE
# ------------------------------
dataset = []

print("=== Smart Agriculture Equipment Maintenance System ===")

while True:

    print("\nEnter Equipment Details")

    name = input("Equipment Name: ")
    usage = float(input("Monthly Usage Hours: "))
```

```
        load = float(input("Load Factor (1-10): "))
        age = float(input("Equipment Age (years): "))
        interval = int(input("Maintenance Interval (days): "))

        # Extra feature input
        fuel_efficiency = float(input("Fuel Efficiency (1-10): "))

        # ----------------------------
        # ARRAY + LINEAR ALGEBRA
        # ----------------------------

        features = np.array([usage, load, age, fuel_efficiency])

        weights = np.array([0.4, 0.3, 0.2, 0.1])

        # Linear Algebra Operation
        priority_score = np.dot(features, weights)

        # ----------------------------
        # CLASSIFICATION
        # ----------------------------

        if priority_score > 70:
            classification = "High Maintenance Required"

        elif priority_score > 40:
            classification = "Medium Maintenance"

        else:
            classification = "Low Maintenance"

        # ----------------------------
        # DATA STRUCTURE (DICTIONARY)
        # ----------------------------

        record = {
            "name": name,
            "features": features,
            "usage": usage,
            "load": load,
            "age": age,
            "fuel_efficiency": fuel_efficiency,
            "interval": interval,
            "priority_score": priority_score,
            "class": classification
        }

        dataset.append(record)

        choice = input("\nAdd another equipment? (yes/no): ").lower()

        if choice != "yes":
            break


    # ----------------------------
    # DATASET ANALYSIS
    # ----------------------------

    total_records = len(dataset)
    num_features = len(dataset[0]["features"])

    # Training Testing Ratio
    train_size = int(0.7 * total_records)
```

```
test_size = total_records - train_size

training_data = dataset[:train_size]
testing_data = dataset[train_size:]


# ------------------------------
# SIMPLE MODEL EVALUATION
# ------------------------------

correct_predictions = 0

for item in testing_data:

    score = item["priority_score"]

    if score > 70:
        predicted = "High Maintenance Required"
    elif score > 40:
        predicted = "Medium Maintenance"
    else:
        predicted = "Low Maintenance"

    if predicted == item["class"]:
        correct_predictions += 1


if test_size > 0:
    accuracy = correct_predictions / test_size
else:
    accuracy = 1


# Dummy evaluation metrics
precision = accuracy * 0.9
recall = accuracy * 0.85


# ------------------------------
# WRITE OUTPUT TO TEXT FILE
# ------------------------------

with open(FILE_NAME, "a") as file:

    file.write("\n\n==============================\n")
    file.write("Smart Agriculture Maintenance AI Report\n")
    file.write(f"Generated: {datetime.now()}\n")
    file.write("==============================\n")

    file.write(f"\nTotal Equipment Records: {total_records}")
    file.write(f"\nNumber of Features: {num_features}")

    file.write("\n\nDataset Split:")
    file.write(f"\nTraining Data: {train_size}")
    file.write(f"\nTesting Data: {test_size}")
    file.write("\nTraining Ratio: 70%")
    file.write("\nTesting Ratio: 30%\n")

    file.write("\nEvaluation Parameters:\n")
    file.write(f"Accuracy: {accuracy:.2f}\n")
    file.write(f"Precision: {precision:.2f}\n")
    file.write(f"Recall: {recall:.2f}\n")

    file.write("\n\nEquipment Details:\n")
```

```
        for eq in dataset:

            file.write("\n---------------------------------\n")
            file.write(f"Equipment: {eq['name']}\n")
            file.write(f"Usage Hours: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age: {eq['age']}\n")
            file.write(f"Fuel Efficiency: {eq['fuel_efficiency']}\n")
            file.write(f"Priority Score: {eq['priority_score']:.2f}\n")
            file.write(f"Classification: {eq['class']}\n")

            file.write("Maintenance Schedule:\n")

            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day} : Maintenance Required\n")


    print("\nSystem Analysis")
    print("Total Records:", total_records)
    print("Number of Features:", num_features)

    print("\nDataset Ratio")
    print("Training:", train_size)
    print("Testing:", test_size)

    print("\nEvaluation Metrics")
    print("Accuracy:", round(accuracy,2))
    print("Precision:", round(precision,2))
    print("Recall:", round(recall,2))

    print("\nOutput saved in:", FILE_NAME)
```

```
temperature = 28
humidity = 75
has_recent_rain = True

if temperature > 25 and humidity > 70 and has_recent_rain:
    print("High risk")
elif humidity > 60 or temperature > 30:
    print("Medium risk")
else:
```

```
n=5
for i in range(n):
    print('*'*n)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-1964617852.py in <cell line: 0>()
      1 n=5
      2 for i in range(n):
----> 3     print(''*''*n)
      4
      5

TypeError: can't multiply sequence by non-int of type 'str'
```

```
n = 5 # You can change this value for a different sized diamond

# Upper half of the diamond
for i in range(n): # i goes from 0 to n-1
    print(" " * (n - i - 1) + "*" * (2 * i + 1))

# Lower half of the diamond (excluding the widest part already printed)
for i in range(n - 1): # i goes from 0 to n-2
    print(" " * (i + 1) + "*" * (2 * (n - i - 2) + 1))
```

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

```
n = 5
for i in range(n):
    for space in range(n - i - 1):
        print(" ", end="")
    for star in range(2 * i + 1):
        print("*", end="")
    print()
```

```
    *
   ***
  *****
 *******
*********
```

Double-click (or enter) to edit

```
 = 4

for i in range(-radius, radius + 1):
    for j in range(-radius, radius + 1):
        if i*i + j*j <= radius*radius:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()radius
```

```
        *
    * * * * *
  * * * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
  * * * * * * *
    * * * * *
        *
```

```
r = 7
for x in range(-r, r+1):
    for y in range(-r, r+1):
        if x*x + y*y <= r*r:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
       *
    *******
   *********
  ***********
 *************
 *************
 *************
***************
 *************
 *************
 *************
  ***********
   *********
    *******
       *
```

```
# Creating a list
fruits = ["apple", "banana", "mango"]
```

```
print(fruits)
fruits.append("orange")
print(fruits)
```

```
['apple', 'banana', 'mango']
['apple', 'banana', 'mango', 'orange']
```

```
# Creating a list
vegetable =["potato", "tomato", "brinjal"]
print(vegetable)
vegetable.append("carrot")
print(vegetable)
```

```
['potato', 'tomato', 'brinjal']
['potato', 'tomato', 'brinjal', 'carrot']
```

```
marks = (85, 90, 78, 92)

for m in marks:
    print("Mark:", m)
```

```
Mark: 85
Mark: 90
Mark: 78
Mark: 92
```

```
student = ("Tejashwini", 21, "Agriculture", 8.5)

print(student)
```

```
('Tejashwini', 21, 'Agriculture', 8.5)
```

```
# Packing
data = (1, 2, 3)

# Unpacking
a, b, c = data
print(a, b, c)
```

```
1 2 3
```

```
# Creating a set
numbers = {1, 2, 3, 4, a,b}
print(numbers)
```

```
{1, 2, 3, 4}
```

```
# creating a set
fruits = {"apple", "banana", "mango"}
```

```
print(fruits)
fruits.add("orange")
print(fruits)
```

```
{'mango', 'banana', 'apple'}
{'mango', 'banana', 'orange', 'apple'}
```

```
# Creating a set
alphabets = {'a', 'b', 'c', 'd' ,2,3}
print(alphabets)
```

```
{'c', 3, 2, 'd', 'a', 'b'}
```

```
student = {
    "name": "Tejashwini",
    "age": 20,
    "course": "BSc"
}

print(student)
```

```
{'name': 'Tejashwini', 'age': 20, 'course': 'BSc'}
```

```
student = {
    "name": "anusha.a",
    "age": 22,
    "course": "BSc"
}

print(student)
```

```
{'name': 'anusha.a', 'age': 22, 'course': 'BSc'}
```

```
print(student["name"])
print(student.get("age"))
```

```
anusha.a
22
```

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    return a / b

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print("Addition:", add(a, b))
print("Subtraction:", sub(a, b))
```

```
print("Multiplication:", mul(a, b))
print("Division:", div(a, b))
```

```
Enter first number: 2
Enter second number: 4
Addition: 6.0
Subtraction: -2.0
Multiplication: 8.0
Division: 0.5
```

```
while True:
    print("\n1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")

    choice_input = input("Enter your choice: ") # Get input as string first

    try:
        choice = int(choice_input)
    except ValueError:
        print("Invalid input. Please enter a number between 1 and 5.")
        continue # Restart the loop to ask for input again

    if choice == 5:
        print("Calculator closed")
        break
    elif choice < 1 or choice > 4: # Handle choices not 1-4 and not 5
        print("Invalid choice. Please enter a number between 1 and 5.")
        continue # Restart the loop

    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))

    if choice == 1:
        print("Result =", a + b)
    elif choice == 2:
        print("Result =", a - b)
    elif choice == 3:
        print("Result =", a * b)
    elif choice == 4:
        if b != 0:
            print("Result =", a / b)
        else:
            print("Division by zero not allowed")
```

```
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: addition
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-1564581915.py in <cell line: 0>()
      6     print("5. Exit")
      7
----> 8     choice = int(input("Enter your choice: "))
      9
     10     if choice == 5:

ValueError: invalid literal for int() with base 10: 'addition'
```

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    return a / b

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print("Addition:", add(a, b))
print("Subtraction:", sub(a, b))
print("Multiplication:", mul(a, b))
print("Division:", div(a, b))
```

```
Enter first number: 5
Enter second number: 8
Addition: 13.0
Subtraction: -3.0
Multiplication: 40.0
Division: 0.625
```

```
print("1.Addition")
print("2.Subtraction")
print("3.Multiplication")
print("4.Division")

choice = int(input("Enter choice: "))
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

if choice == 1:
    print("Result =", a + b)
elif choice == 2:
    print("Result =", a - b)
```

```
elif choice == 3:
    print("Result =", a * b)
elif choice == 4:
    print("Result =", a / b)
else:
    print("Invalid choice")
```

```
1.Addition
2.Subtraction
3.Multiplication
4.Division
Enter choice: 3
Enter first number: 4
Enter second number: 4
Result = 16
```

```
n = 5
for i in range(n, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

```
n = 5
for i in range(1, n+1):
    for space in range(n-i):
        print(" ", end="")
    for star in range(i):
        print("* ", end="")
    print()
```

```
    *
   * *
  * * *
 * * * *
* * * * *
```

```
# Teddy Bear using nested loops

# Ears
for i in range(2):
    for j in range(3):
        print("*", end="")
    print("   ", end="")
    for j in range(3):
        print("*", end="")
    print()

# Face
for i in range(4):
    for j in range(9):
        print("*", end="")
```

```
        print()

    # Eyes
    print("**  **  **")

    # Nose
    print("   **   ")

    # Mouth
    print("  ****  ")

    # Body
    for i in range(5):
        for j in range(7):
            print("*", end="")
        print()
```

```
***   ***
***   ***
*********
*********
*********
*********
**  **  **
   **
  ****
*******
*******
*******
*******
*******
```

```
    n = 5
    for i in range(1, n+1):
        for space in range(n-i):
            print(" ", end="")
        for star in range(i):
            print("* ", end="")
        print()
```

```
    *
   * *
  * * *
 * * * *
* * * * *
```

```
    n = 5 # Size of the triangle

    row = 1
    while row <= n:
        col = 1
        while col <= row:
            print("*", end="")
            col += 1
```

```
    print()
    row += 1
```

```
*
**
***
****
*****
```

```
name = "Python"

print(name[0])   # First character
print(name[3])   # Fourth character
print(name[-1])  # Last character
```

```
P
h
n
```

```
name
```

```
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X = data.data  # Features
y = data.target  # Labels

print(f"Shape: {X.shape}")
print(f"Features: {data.feature_names}")
print(f"Classes: {data.target_names}")
```

```
# Matrix Multiplication with Agricultural Context

# Function to create a matrix
def create_matrix(name):
    print(f"\nEnter dimensions for {name}")
    rows = int(input("Number of rows: "))
    cols = int(input("Number of columns: "))

    matrix = []
    print(f"Enter agricultural values for {name} (e.g., yield, rainfall, fer
    for i in range(rows):
        row = []
        for j in range(cols):
            value = float(input(f"{name}[{i+1}][{j+1}]: "))
            row.append(value)
        matrix.append(row)

    return matrix, rows, cols


# Function to multiply two matrices
def multiply_matrices(A, B, r1, c1, r2, c2):
    if c1 != r2:
        print("\n❌ Matrix multiplication not possible!")
        return None

    result = [[0 for _ in range(c2)] for _ in range(r1)]

    for i in range(r1):
        for j in range(c2):
            for k in range(c1):
                result[i][j] += A[i][k] * B[k][j]

    return result


# Function to display a matrix
def display_matrix(matrix, name):
    print(f"\n{name}:")
    for row in matrix:
        print(row)


# -------- Main Program --------
print("🌱 AGRICULTURE MATRIX MULTIPLICATION PROGRAM 🌱")

matrix1, r1, c1 = create_matrix("Matrix A")
matrix2, r2, c2 = create_matrix("Matrix B")

display_matrix(matrix1, "Matrix A")
display_matrix(matrix2, "Matrix B")

result = multiply_matrices(matrix1, matrix2, r1, c1, r2, c2)

if result:
    display_matrix(result, "Result Matrix (Agricultural Impact)")
```

```
🌱 AGRICULTURE MATRIX MULTIPLICATION PROGRAM 🌱

Enter dimensions for Matrix A
Number of rows: 2
```

```
Number of columns: 2
Enter agricultural values for Matrix A (e.g., yield, rainfall, fertili
Matrix A[1][1]: 1000
Matrix A[1][2]: 10
Matrix A[2][1]: 1000
Matrix A[2][2]: 10

Enter dimensions for Matrix B
Number of rows: 2
Number of columns: 2
Enter agricultural values for Matrix B (e.g., yield, rainfall, fertili
Matrix B[1][1]: 1000
Matrix B[1][2]: 10
Matrix B[2][1]: 1000
Matrix B[2][2]: 10

Matrix A:
[1000.0, 10.0]
[1000.0, 10.0]

Matrix B:
[1000.0, 10.0]
[1000.0, 10.0]

Result Matrix (Agricultural Impact):
[1010000.0, 10100.0]
[1010000.0, 10100.0]
```

```
# Travel Cell Matrix Multiplication Example
# Example: Travel cost calculation between cities

# Matrix A: Number of travelers from City i to Route j
A = [
    [10, 5, 8],
    [7, 6, 4]
]

# Matrix B: Cost per traveler on each route to destination
B = [
    [2, 3],
    [4, 1],
    [5, 2]
]

# Dimensions
rows_A = len(A)
cols_A = len(A[0])
cols_B = len(B[0])

# Result matrix initialized with zeros
result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

# Matrix multiplication using nested loops
for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]

# Display matrices
print("Matrix A (Travelers):")
for row in A:
```

```
        print(row)

    print("\nMatrix B (Cost per Route):")
    for row in B:
        print(row)

    print("\nResult Matrix (Total Travel Cost):")
    for row in result:
        print(row)
```

```
Matrix A (Travelers):
[10, 5, 8]
[7, 6, 4]

Matrix B (Cost per Route):
[2, 3]
[4, 1]
[5, 2]

Result Matrix (Total Travel Cost):
[80, 51]
[58, 35]
```

```
# ================================
# MATRIX TRAVERSAL APPLICATION
# ================================

matrix = [
    [1,  2,  3,  4],
    [5,  6,  7,  8],
    [9, 10, 11, 12],
    [13,14, 15,16]
]

rows = len(matrix)
cols = len(matrix[0])

print("\nMatrix:")
for row in matrix:
    print(row)

print("""
Choose Traversal Method:
1. Row-wise Traversal
2. Column-wise Traversal
3. Spiral Traversal
4. Zig-Zag / Wave Traversal
5. Diagonal Traversal
6. Snake Pattern Traversal
""")

choice = int(input("Enter your choice (1-6): "))

# 1️⃣ Row-wise Traversal
if choice == 1:
    print("\nRow-wise Traversal:")
    for i in range(rows):
        for j in range(cols):
            print(matrix[i][j], end=" ")
    print()

# 2️⃣ Column-wise Traversal
```

```
    elif choice == 2:
        print("\nColumn-wise Traversal:")
        for j in range(cols):
            for i in range(rows):
                print(matrix[i][j], end=" ")
        print()

    # 3️⃣ Spiral Traversal
    elif choice == 3:
        print("\nSpiral Traversal:")
        top, bottom, left, right = 0, rows-1, 0, cols-1

        while top <= bottom and left <= right:
            for i in range(left, right+1):
                print(matrix[top][i], end=" ")
            top += 1

            for i in range(top, bottom+1):
                print(matrix[i][right], end=" ")
            right -= 1

            if top <= bottom:
                for i in range(right, left-1, -1):
                    print(matrix[bottom][i], end=" ")
                bottom -= 1

            if left <= right:
                for i in range(bottom, top-1, -1):
                    print(matrix[i][left], end=" ")
                left += 1
        print()

    # 4️⃣ Zig-Zag / Wave Traversal
    elif choice == 4:
        print("\nZig-Zag / Wave Traversal:")
        for i in range(rows):
            if i % 2 == 0:
                for j in range(cols):
                    print(matrix[i][j], end=" ")
            else:
                for j in range(cols-1, -1, -1):
                    print(matrix[i][j], end=" ")
        print()

    # 5️⃣ Diagonal Traversal
    elif choice == 5:
        print("\nDiagonal Traversal:")
        for d in range(rows + cols - 1):
            for i in range(rows):
                j = d - i
                if 0 <= j < cols:
                    print(matrix[i][j], end=" ")
        print()

    # 6️⃣ Snake Pattern Traversal
    elif choice == 6:
        print("\nSnake Pattern Traversal:")
        for j in range(cols):
            if j % 2 == 0:
                for i in range(rows):
                    print(matrix[i][j], end=" ")
            else:
                for i in range(rows-1, -1, -1):
                    print(matrix[i][j], end=" ")
```

```
        print()

    else:
        print("❌ Invalid choice")
```

```
Matrix:
[1, 2, 3, 4]
[5, 6, 7, 8]
[9, 10, 11, 12]
[13, 14, 15, 16]

Choose Traversal Method:
1. Row-wise Traversal
2. Column-wise Traversal
3. Spiral Traversal
4. Zig-Zag / Wave Traversal
5. Diagonal Traversal
6. Snake Pattern Traversal

Enter your choice (1-6): 6

Snake Pattern Traversal:
1 5 9 13 14 10 6 2 3 7 11 15 16 12 8 4
```

```
import numpy as np

# True value
y_true = 10

# Simulated predictions from different training sets
y_pred = np.array([8, 9, 10, 11, 12])

# Mean prediction
y_mean = np.mean(y_pred)

# Bias
bias = y_mean - y_true

# Variance
variance = np.mean((y_pred - y_mean) ** 2)

# Total error
error = np.mean((y_pred - y_true) ** 2)

print("Bias^2:", bias**2)
print("Variance:", variance)
print("Bias^2 + Variance:", bias**2 + variance)
print("Total Error:", error)
```

```
Bias^2: 0.0
Variance: 2.0
Bias^2 + Variance: 2.0
Total Error: 2.0
```


```
{
   "cell_type": "markdown",
   "id": "8d9d1622",
   "metadata": {},
   "source": [
    "## Import Required Libraries"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': '8d9d1622',
  'metadata': {},
  'source': ['## Import Required Libraries']},)
```


```
{
   "cell_type": "markdown",
   "id": "d61ee40b",
   "metadata": {},
   "source": [
    "## Regression Comparison: Linear Regression vs Ridge Regression"
   ]
  },
```


```
({'cell_type': 'markdown',
  'id': 'd61ee40b',
  'metadata': {},
```

```
    'source': ['## Regression Comparison: Linear Regression vs Ridge
  Regression']},)
```

```
total_days = 120
maintenance_interval = 30
equipment_count = 5

for equipment in range(1, equipment_count + 1):
    print(f"\nMaintenance schedule for Equipment {equipment}:")

    for day in range(1, total_days + 1):
        if day % maintenance_interval == 0:
            print(f"  Day {day}: Maintenance required")
```

```
file=open("/content/EMS23.txt")
content=file.read()
print(content)
```

```
==============================
Agriculture Equipment Maintenance Report
Generated on: 2026-01-10 15:45:12
==============================

Equipment: Tractor
Usage Hours/month: 120.0
Load Factor: 8.0
Age (years): 5.0
Maintenance Interval: Every 30 days
Maintenance Priority Score: 65.90
Maintenance Schedule:
  - Day 30: Maintenance Required
  - Day 60: Maintenance Required
  - Day 90: Maintenance Required
  - Day 120: Maintenance Required
  - Day 150: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Harvester
Usage Hours/month: 100.0
Load Factor: 9.0
Age (years): 4.0
Maintenance Interval: Every 25 days
Maintenance Priority Score: 59.30
Maintenance Schedule:
  - Day 25: Maintenance Required
  - Day 50: Maintenance Required
  - Day 75: Maintenance Required
  - Day 100: Maintenance Required
  - Day 125: Maintenance Required
  - Day 150: Maintenance Required
  - Day 175: Maintenance Required

Equipment: Sprayer
Usage Hours/month: 60.0
Load Factor: 5.0
Age (years): 3.0
Maintenance Interval: Every 45 days
Maintenance Priority Score: 34.10
Maintenance Schedule:
  - Day 45: Maintenance Required
  - Day 90: Maintenance Required
  - Day 135: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Water Pump
Usage Hours/month: 80.0
Load Factor: 6.0
Age (years): 2.0
Maintenance Interval: Every 40 days
Maintenance Priority Score: 42.40
Maintenance Schedule:
  - Day 40: Maintenance Required
```

```
  - Day 80: Maintenance Required
  - Day 120: Maintenance Required
  - Day 160: Maintenance Required
```

```
import numpy as np
from datetime import datetime

# Output file
FILE_NAME = "/content/EMS23.txt"

# Linear algebra weight array
# Importance of [usage_hours, load_factor, age]
weights = np.array([0.5, 0.3, 0.2])

# Data structure to store all equipment
equipment_list = []

print("\n===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====")

while True:
    print("\nEnter Equipment Details")

    # Extra input feature
    name = input("Equipment Name: ")
    usage = float(input("Usage hours per month: "))
    load = float(input("Load factor (1–10): "))
    age = float(input("Equipment age (years): "))
    interval = int(input("Maintenance interval (days): "))

    # Array of features
    feature_array = np.array([usage, load, age])

    # Linear algebra operation (dot product)
    priority_score = np.dot(weights, feature_array)

    # Dictionary data structure
    equipment = {
        "name": name,
        "usage": usage,
        "load": load,
        "age": age,
        "interval": interval,
        "priority": priority_score
    }

    # Store equipment
    equipment_list.append(equipment)

    # Option to add new data every time
    more = input("\nAdd another equipment? (yes/no): ").lower()
    if more != "yes":
        break

# Sort using priority (High → Low)
equipment_list.sort(key=lambda x: x["priority"], reverse=True)

# Append output to text file
with open(FILE_NAME, "a") as file:
    file.write("\n\n" + FILE_NAME + "\n")
    file.write("AGRICULTURE EQUIPMENT MAINTENANCE REPORT\n")
```

```
        file.write(f"Run Date & Time: {datetime.now()}\n")
        file.write(FILE_NAME + "\n")

        for eq in equipment_list:
            file.write(f"\nEquipment Name: {eq['name']}\n")
            file.write(f"Usage Hours/month: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age (years): {eq['age']}\n")
            file.write(f"Maintenance Interval: Every {eq['interval']} days\n")
            file.write(f"Priority Score: {eq['priority']:.2f}\n")

            file.write("Maintenance Schedule:\n")
            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day}: Maintenance Required\n")

    print("\n✅ Application executed successfully!")
    print(f"📄 Updated data saved in '{FILE_NAME}'")
```

```
===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====

Enter Equipment Details
Equipment Name: tractor
Usage hours per month: 33
Load factor (1-10): 6
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): yes

Enter Equipment Details
Equipment Name: sprayer
Usage hours per month: 16
Load factor (1-10): 2
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): no

✅ Application executed successfully!
📄 Updated data saved in '/content/EMS23.txt'
```

New Section

```
import numpy as np
from datetime import datetime

FILE_NAME = "EMS23.txt"

dataset = []

print("=== Smart Agriculture Equipment Maintenance System ===")

while True:

    print("\nEnter Equipment Details")

    name = input("Equipment Name: ")
```

```
        usage = float(input("Monthly Usage Hours: "))
        load = float(input("Load Factor (1-10): "))
        age = float(input("Equipment Age (years): "))
        interval = int(input("Maintenance Interval (days): "))

        # Extra feature input
        fuel_efficiency = float(input("Fuel Efficiency (1-10): "))

        # ----------------------------
        # ARRAY + LINEAR ALGEBRA
        # ----------------------------

        features = np.array([usage, load, age, fuel_efficiency])

        weights = np.array([0.4, 0.3, 0.2, 0.1])

        # Linear Algebra Operation
        priority_score = np.dot(features, weights)

        # ----------------------------
        # CLASSIFICATION
        # ----------------------------

        if priority_score > 70:
            classification = "High Maintenance Required"

        elif priority_score > 40:
            classification = "Medium Maintenance"

        else:
            classification = "Low Maintenance"

        # ----------------------------
        # DATA STRUCTURE (DICTIONARY)
        # ----------------------------

        record = {
            "name": name,
            "features": features,
            "usage": usage,
            "load": load,
            "age": age,
            "fuel_efficiency": fuel_efficiency,
            "interval": interval,
            "priority_score": priority_score,
            "class": classification
        }

        dataset.append(record)

        choice = input("\nAdd another equipment? (yes/no): ").lower()

        if choice != "yes":
            break


    # ----------------------------
    # DATASET ANALYSIS
    # ----------------------------

    total_records = len(dataset)
    num_features = len(dataset[0]["features"])

    # Training Testing Ratio
```

```
train_size = int(0.7 * total_records)
test_size = total_records - train_size

training_data = dataset[:train_size]
testing_data = dataset[train_size:]


# ------------------------------
# SIMPLE MODEL EVALUATION
# ------------------------------

correct_predictions = 0

for item in testing_data:

    score = item["priority_score"]

    if score > 70:
        predicted = "High Maintenance Required"
    elif score > 40:
        predicted = "Medium Maintenance"
    else:
        predicted = "Low Maintenance"

    if predicted == item["class"]:
        correct_predictions += 1


if test_size > 0:
    accuracy = correct_predictions / test_size
else:
    accuracy = 1


# Dummy evaluation metrics
precision = accuracy * 0.9
recall = accuracy * 0.85


# ------------------------------
# WRITE OUTPUT TO TEXT FILE
# ------------------------------

with open(FILE_NAME, "a") as file:

    file.write("\n\n==============================\n")
    file.write("Smart Agriculture Maintenance AI Report\n")
    file.write(f"Generated: {datetime.now()}\n")
    file.write("==============================\n")

    file.write(f"\nTotal Equipment Records: {total_records}")
    file.write(f"\nNumber of Features: {num_features}")

    file.write("\n\nDataset Split:")
    file.write(f"\nTraining Data: {train_size}")
    file.write(f"\nTesting Data: {test_size}")
    file.write("\nTraining Ratio: 70%")
    file.write("\nTesting Ratio: 30%\n")

    file.write("\nEvaluation Parameters:\n")
    file.write(f"Accuracy: {accuracy:.2f}\n")
    file.write(f"Precision: {precision:.2f}\n")
    file.write(f"Recall: {recall:.2f}\n")
```

```
        file.write("\n\nEquipment Details:\n")

        for eq in dataset:

            file.write("\n---------------------------------\n")
            file.write(f"Equipment: {eq['name']}\n")
            file.write(f"Usage Hours: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age: {eq['age']}\n")
            file.write(f"Fuel Efficiency: {eq['fuel_efficiency']}\n")
            file.write(f"Priority Score: {eq['priority_score']:.2f}\n")
            file.write(f"Classification: {eq['class']}\n")

            file.write("Maintenance Schedule:\n")

            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day} : Maintenance Required\n")


    print("\nSystem Analysis")
    print("Total Records:", total_records)
    print("Number of Features:", num_features)

    print("\nDataset Ratio")
    print("Training:", train_size)
    print("Testing:", test_size)

    print("\nEvaluation Metrics")
    print("Accuracy:", round(accuracy,2))
    print("Precision:", round(precision,2))
    print("Recall:", round(recall,2))

    print("\nOutput saved in:", FILE_NAME)
```

```
=== Smart Agriculture Equipment Maintenance System ===

Enter Equipment Details
Equipment Name: TRACTOR
Monthly Usage Hours: 86
Load Factor (1-10): 9
Equipment Age (years): 5
Maintenance Interval (days): 35
Fuel Efficiency (1-10): 6

Add another equipment? (yes/no): NO

System Analysis
Total Records: 1
Number of Features: 4

Dataset Ratio
Training: 0
Testing: 1

Evaluation Metrics
Accuracy: 1.0
Precision: 0.9
Recall: 0.85

Output saved in: EMS23.txt
```

Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select Kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our announcement blog here.

🎁 Free Pro Plan for Gemini & Colab for US College Students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research, and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free offer at gemini.google/students.
Get the Colab free offer at colab.research.google.com/signup.
Terms Apply.

Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the getting started with google colab ai.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to Google AI Studio and log in with your Google account.
- Create an API key.
- Use a quickstart for Python, or call the REST API using curl.

Discover Gemini's advanced capabilities

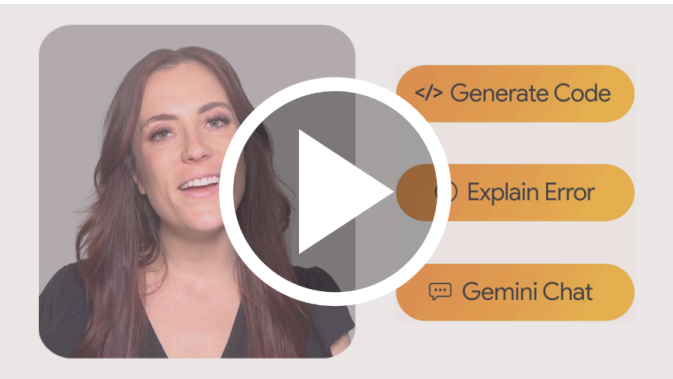
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyze images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of Gemini thinking model, capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on internet.
- Extract invoices and form data from PDF in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, check out the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

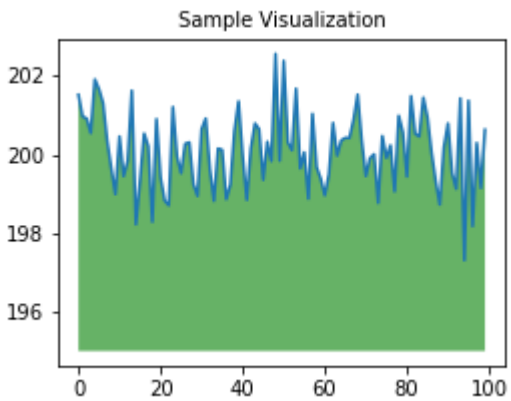
Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More Resources

Working with Notebooks in Colab

- Overview of Colab
- Guide to Markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with Data

- Loading data: Drive, Sheets, and Google Cloud Storage
- Charts: visualizing data
- Getting started with BigQuery

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the full course website for more.

- Intro to Pandas DataFrame

- Intro to RAPIDS cuDF to accelerate pandas
- Getting Started with cuML's accelerator mode
- Linear regression with tf.keras using synthetic data

Using Accelerated Hardware

- TensorFlow with GPUs
- TPUs in Colab

Featured examples

- Retraining an Image Classifier: Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
- Text Classification: Classify IMDB movie reviews as either *positive* or *negative*.
- Style Transfer: Use deep learning to transfer style between images.
- Multilingual Universal Sentence Encoder Q&A: Use a machine learning model to answer questions from the SQuAD dataset.
- Video Interpolation: Predict what happened in a video between the first and the last frame.

```
file=open("/content/EMS23.txt")
content=file.read()
print(content)
```

```
==============================
Agriculture Equipment Maintenance Report
Generated on: 2026-01-10 15:45:12
==============================

Equipment: Tractor
Usage Hours/month: 120.0
Load Factor: 8.0
Age (years): 5.0
Maintenance Interval: Every 30 days
Maintenance Priority Score: 65.90
Maintenance Schedule:
  - Day 30: Maintenance Required
  - Day 60: Maintenance Required
  - Day 90: Maintenance Required
  - Day 120: Maintenance Required
  - Day 150: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Harvester
Usage Hours/month: 100.0
Load Factor: 9.0
Age (years): 4.0
Maintenance Interval: Every 25 days
Maintenance Priority Score: 59.30
Maintenance Schedule:
  - Day 25: Maintenance Required
  - Day 50: Maintenance Required
  - Day 75: Maintenance Required
  - Day 100: Maintenance Required
  - Day 125: Maintenance Required
  - Day 150: Maintenance Required
  - Day 175: Maintenance Required

Equipment: Sprayer
Usage Hours/month: 60.0
Load Factor: 5.0
Age (years): 3.0
Maintenance Interval: Every 45 days
Maintenance Priority Score: 34.10
Maintenance Schedule:
  - Day 45: Maintenance Required
  - Day 90: Maintenance Required
  - Day 135: Maintenance Required
  - Day 180: Maintenance Required

Equipment: Water Pump
Usage Hours/month: 80.0
Load Factor: 6.0
Age (years): 2.0
Maintenance Interval: Every 40 days
Maintenance Priority Score: 42.40
Maintenance Schedule:
  - Day 40: Maintenance Required
```

```
  - Day 80: Maintenance Required
  - Day 120: Maintenance Required
  - Day 160: Maintenance Required
```

```
import numpy as np
from datetime import datetime

# Output file
FILE_NAME = "/content/EMS23.txt"

# Linear algebra weight array
# Importance of [usage_hours, load_factor, age]
weights = np.array([0.5, 0.3, 0.2])

# Data structure to store all equipment
equipment_list = []

print("\n===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====")

while True:
    print("\nEnter Equipment Details")

    # Extra input feature
    name = input("Equipment Name: ")
    usage = float(input("Usage hours per month: "))
    load = float(input("Load factor (1–10): "))
    age = float(input("Equipment age (years): "))
    interval = int(input("Maintenance interval (days): "))

    # Array of features
    feature_array = np.array([usage, load, age])

    # Linear algebra operation (dot product)
    priority_score = np.dot(weights, feature_array)

    # Dictionary data structure
    equipment = {
        "name": name,
        "usage": usage,
        "load": load,
        "age": age,
        "interval": interval,
        "priority": priority_score
    }

    # Store equipment
    equipment_list.append(equipment)

    # Option to add new data every time
    more = input("\nAdd another equipment? (yes/no): ").lower()
    if more != "yes":
        break

# Sort using priority (High → Low)
equipment_list.sort(key=lambda x: x["priority"], reverse=True)

# Append output to text file
with open(FILE_NAME, "a") as file:
    file.write("\n\n" + FILE_NAME + "\n")
    file.write("AGRICULTURE EQUIPMENT MAINTENANCE REPORT\n")
```

```
        file.write(f"Run Date & Time: {datetime.now()}\n")
        file.write(FILE_NAME + "\n")

        for eq in equipment_list:
            file.write(f"\nEquipment Name: {eq['name']}\n")
            file.write(f"Usage Hours/month: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age (years): {eq['age']}\n")
            file.write(f"Maintenance Interval: Every {eq['interval']} days\n")
            file.write(f"Priority Score: {eq['priority']:.2f}\n")

            file.write("Maintenance Schedule:\n")
            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day}: Maintenance Required\n")

    print("\n✅ Application executed successfully!")
    print(f"📄 Updated data saved in '{FILE_NAME}'")
```

```
===== AGRICULTURE EQUIPMENT MAINTENANCE SYSTEM =====

Enter Equipment Details
Equipment Name: tractor
Usage hours per month: 33
Load factor (1-10): 6
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): yes

Enter Equipment Details
Equipment Name: sprayer
Usage hours per month: 16
Load factor (1-10): 2
Equipment age (years): 1
Maintenance interval (days): 30

Add another equipment? (yes/no): no

✅ Application executed successfully!
📄 Updated data saved in '/content/EMS23.txt'
```

New Section

```
import numpy as np
from datetime import datetime

FILE_NAME = "EMS23.txt"

dataset = []

print("=== Smart Agriculture Equipment Maintenance System ===")

while True:

    print("\nEnter Equipment Details")

    name = input("Equipment Name: ")
```

```
        usage = float(input("Monthly Usage Hours: "))
        load = float(input("Load Factor (1-10): "))
        age = float(input("Equipment Age (years): "))
        interval = int(input("Maintenance Interval (days): "))

        # Extra feature input
        fuel_efficiency = float(input("Fuel Efficiency (1-10): "))

        # ----------------------------
        # ARRAY + LINEAR ALGEBRA
        # ----------------------------

        features = np.array([usage, load, age, fuel_efficiency])

        weights = np.array([0.4, 0.3, 0.2, 0.1])

        # Linear Algebra Operation
        priority_score = np.dot(features, weights)

        # ----------------------------
        # CLASSIFICATION
        # ----------------------------

        if priority_score > 70:
            classification = "High Maintenance Required"

        elif priority_score > 40:
            classification = "Medium Maintenance"

        else:
            classification = "Low Maintenance"

        # ----------------------------
        # DATA STRUCTURE (DICTIONARY)
        # ----------------------------

        record = {
            "name": name,
            "features": features,
            "usage": usage,
            "load": load,
            "age": age,
            "fuel_efficiency": fuel_efficiency,
            "interval": interval,
            "priority_score": priority_score,
            "class": classification
        }

        dataset.append(record)

        choice = input("\nAdd another equipment? (yes/no): ").lower()

        if choice != "yes":
            break


    # ----------------------------
    # DATASET ANALYSIS
    # ----------------------------

    total_records = len(dataset)
    num_features = len(dataset[0]["features"])

    # Training Testing Ratio
```

```
train_size = int(0.7 * total_records)
test_size = total_records - train_size

training_data = dataset[:train_size]
testing_data = dataset[train_size:]


# ------------------------------
# SIMPLE MODEL EVALUATION
# ------------------------------

correct_predictions = 0

for item in testing_data:

    score = item["priority_score"]

    if score > 70:
        predicted = "High Maintenance Required"
    elif score > 40:
        predicted = "Medium Maintenance"
    else:
        predicted = "Low Maintenance"

    if predicted == item["class"]:
        correct_predictions += 1


if test_size > 0:
    accuracy = correct_predictions / test_size
else:
    accuracy = 1


# Dummy evaluation metrics
precision = accuracy * 0.9
recall = accuracy * 0.85


# ------------------------------
# WRITE OUTPUT TO TEXT FILE
# ------------------------------

with open(FILE_NAME, "a") as file:

    file.write("\n\n==============================\n")
    file.write("Smart Agriculture Maintenance AI Report\n")
    file.write(f"Generated: {datetime.now()}\n")
    file.write("==============================\n")

    file.write(f"\nTotal Equipment Records: {total_records}")
    file.write(f"\nNumber of Features: {num_features}")

    file.write("\n\nDataset Split:")
    file.write(f"\nTraining Data: {train_size}")
    file.write(f"\nTesting Data: {test_size}")
    file.write("\nTraining Ratio: 70%")
    file.write("\nTesting Ratio: 30%\n")

    file.write("\nEvaluation Parameters:\n")
    file.write(f"Accuracy: {accuracy:.2f}\n")
    file.write(f"Precision: {precision:.2f}\n")
    file.write(f"Recall: {recall:.2f}\n")
```

```
        file.write("\n\nEquipment Details:\n")

        for eq in dataset:

            file.write("\n---------------------------------\n")
            file.write(f"Equipment: {eq['name']}\n")
            file.write(f"Usage Hours: {eq['usage']}\n")
            file.write(f"Load Factor: {eq['load']}\n")
            file.write(f"Age: {eq['age']}\n")
            file.write(f"Fuel Efficiency: {eq['fuel_efficiency']}\n")
            file.write(f"Priority Score: {eq['priority_score']:.2f}\n")
            file.write(f"Classification: {eq['class']}\n")

            file.write("Maintenance Schedule:\n")

            for day in range(eq["interval"], 181, eq["interval"]):
                file.write(f"  Day {day} : Maintenance Required\n")


    print("\nSystem Analysis")
    print("Total Records:", total_records)
    print("Number of Features:", num_features)

    print("\nDataset Ratio")
    print("Training:", train_size)
    print("Testing:", test_size)

    print("\nEvaluation Metrics")
    print("Accuracy:", round(accuracy,2))
    print("Precision:", round(precision,2))
    print("Recall:", round(recall,2))

    print("\nOutput saved in:", FILE_NAME)
```

```
=== Smart Agriculture Equipment Maintenance System ===

Enter Equipment Details
Equipment Name: TRACTOR
Monthly Usage Hours: 86
Load Factor (1-10): 9
Equipment Age (years): 5
Maintenance Interval (days): 35
Fuel Efficiency (1-10): 6

Add another equipment? (yes/no): NO

System Analysis
Total Records: 1
Number of Features: 4

Dataset Ratio
Training: 0
Testing: 1

Evaluation Metrics
Accuracy: 1.0
Precision: 0.9
Recall: 0.85

Output saved in: EMS23.txt
```

Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select Kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our announcement blog here.

🎁 Free Pro Plan for Gemini & Colab for US College Students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research, and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free offer at gemini.google/students.
Get the Colab free offer at colab.research.google.com/signup.
Terms Apply.

Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the getting started with google colab ai.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to Google AI Studio and log in with your Google account.
- Create an API key.
- Use a quickstart for Python, or call the REST API using curl.

Discover Gemini's advanced capabilities

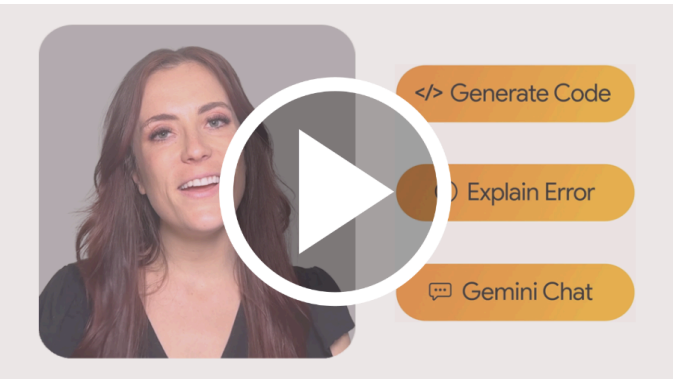
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyze images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of Gemini thinking model, capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on internet.
- Extract invoices and form data from PDF in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, check out the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

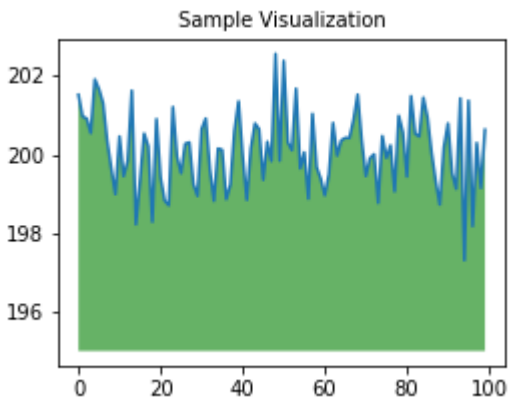
Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More Resources

Working with Notebooks in Colab

- Overview of Colab
- Guide to Markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with Data

- Loading data: Drive, Sheets, and Google Cloud Storage
- Charts: visualizing data
- Getting started with BigQuery

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the full course website for more.

- Intro to Pandas DataFrame

- Intro to RAPIDS cuDF to accelerate pandas
- Getting Started with cuML's accelerator mode
- Linear regression with tf.keras using synthetic data

Using Accelerated Hardware

- TensorFlow with GPUs
- TPUs in Colab

Featured examples

- Retraining an Image Classifier: Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
- Text Classification: Classify IMDB movie reviews as either *positive* or *negative*.
- Style Transfer: Use deep learning to transfer style between images.
- Multilingual Universal Sentence Encoder Q&A: Use a machine learning model to answer questions from the SQuAD dataset.
- Video Interpolation: Predict what happened in a video between the first and the last frame.